

ENT207TC Digital Startup Lab | Phase 2: Building | Week 4 Deliverable

From Footprint to Build

Choosing the LineSight Technology Stack

Checkpoint Draft Workshop — Choose Stack: candidate comparison, weighted decision, technology profiles with lineage, and the build plan to Checkpoint 2



Chosen: Flutter + Dart for Android and iOS | React + TypeScript for desktop and mobile web

Document control

Field	Value
Document	LineSight Week 4 — Technology Stack Decision (Choose Stack workshop output)
Revision	1.0 — workshop draft, issued for mentor review
Team	LineSight (ENT207TC group); roles listed in §14.1
Date	7 October 2026
Upstream inputs	Checkpoint 1 pack (Week 3): validated problem, solution concept, clickable digital prototype; Week 2 product deliverable §9–§11 (screens, data logic, three-layer architecture)
Downstream consumers	Draft 1 (Digital Prototype Report), Checkpoint 2 report (Week 7: tech stack rationale + working app with three features), Digital Prototype Report (Week 10)
Decision status	Adopted for the build phase , with the two kill criteria in §12.3 and the Week 4 spikes in §13.2 still to be cleared
Method	Weighted multi-criteria scoring, 9 criteria, 10 app-side and 9 web-side candidates, 3 independent scorers, sensitivity analysis on all weights

How to read this document

Sections 1–4 establish why the stack question is hard for *this* product: what Checkpoint 1 locked in, what the four delivery targets actually demand, and which requirements were promoted to decision drivers. Section 5 defines the scoring instrument before any candidate is named, so that the arithmetic in §11 can be audited. Sections 6–9 are the market survey: the longest, then profiles of every serious candidate — what it is, where it came from, what it is genuinely good at, and where it hurts. Section 10 covers the layers behind the two headline choices (data, backend, inference, agents, delivery), which is where most student projects actually fail. Sections 11–14 contain the decision itself: scores, sensitivity, architecture, risks, decision records and the build plan to Checkpoint 2. Appendices carry the raw scoring sheet, the logo and licence attribution, and the glossary.

Executive summary

[DECISION] Mobile (Android + iOS): Flutter with Dart. Web (desktop browser + mobile browser): React with TypeScript, built by Vite as a single-page application. Behind those two choices sit eight supporting decisions — SQLite-plus-outbox at the edge, PostgreSQL plus a Python (FastAPI) service at the centre, ONNX Runtime for inference, a server-side agent runtime speaking Model Context Protocol to our own inspection API, GitHub Actions plus Codemagic for delivery, and Sentry/PostHog for evidence — all recorded as ADRs in §14.

[RESULT] Ten app-side candidates were scored against nine weighted criteria. Flutter finished first at **4.14/5.00**, with React Native + Expo second at 4.09 — a gap of 0.055, which we treat as a tie and resolve on other grounds. Ionic + Capacitor third at 3.74, the two-native-codebases option fourth at 3.56, Kotlin Multiplatform sixth at 3.45, and the three delivery-only routes (NativeScript 2.78, low-code 2.77, .NET MAUI 2.73) at the bottom. The full arithmetic, including the two weight perturbations that change the order, is in §11.

[TENSION] The honest tension in this decision is that the team's existing skill asset — React and TypeScript, from the Week 2 prototype — points at React Native, while the product's hardest requirement — a 60 fps painted verdict surface with camera overlays on a \$120 line-side Android panel, used with gloves — points at Flutter. We resolved it by splitting the surfaces rather

than compromising on one: the operator-facing HMI is where the product earns its existence, so it takes the framework that paints best; the engineering-facing web surfaces are where the team's existing skill pays back fastest, so they stay in React. One codebase per persona, one design-token source, one API contract.

[PLAN] Weeks 4–7 deliver, in order: five time-boxed stack spikes with numeric pass gates (§13.2); Feature 1 Live View on a real panel with a real event store (Week 5); Feature 2 Review and traceability with export, plus the web dashboard (Week 6); Feature 3 recipe changeover with the guided checklist and the first agent surface (Week 7), which is also Checkpoint 2. Two kill criteria in §12.3 define the point at which we abandon Flutter rather than defend it.

Surface	Target	Stack	Rationale in one line
Operator HMI	Android panel / tablet, iOS	Flutter (Dart)	Best painted-frame budget, one codebase for both handsets and the 7-inch panel, first-class camera and on-device inference paths
Engineer / supervisor view	Desktop browser	React 19 + TypeScript (Vite SPA)	Continuity with the Week 2 prototype, richest data-table and chart ecosystem, static hosting inside the plant LAN
Field view	Mobile browser (PWA)	Same React codebase, responsive + service worker	One web codebase serves both browser sizes; installable, offline shell
Centre	Plant server / cloud	Python FastAPI + PostgreSQL + Docker	Reuses the Week 2 simulation and inference code; typed OpenAPI contract feeds both front ends
Edge store	On-device	SQLite (Drift) + outbox sync	The Week 2 §10.1 event schema lands unchanged; survives the network
Agents	Server-side	MCP tools over our own REST + DeepSeek/Qwen-class models	Week 4 workshop requirement; keeps model choice swappable and the audit trail server-side

Table 1 — Decision at a glance. Every row is defended in the section named at its head.

1 | From footprint to build: what Week 4 actually changes

[CONTEXT] Checkpoints 1–3 left LineSight as a *demonstrated footprint*: a validated problem (blind manual inspection at the line-side station, with published correct-reject rates of only about 70–85%), a solution concept, five wireframed touch-first screens, a clickable design prototype, and a complete event data model whose schema was fixed in the Week 2 deliverable §10.1. Nothing in that package required a compiler. Week 4 is the first week in which the artefact is a program: it must run on hardware we do not fully control, in a network we cannot assume is up, in front of a person wearing gloves, and it must still be running at Demo Day in Week 10.

[CONTEXT] That transition changes the nature of the decision. A wireframe tool choice is reversible in an afternoon; a framework choice is reversible in about a month of team life, which is more than a third of the time we have left. The module handbook is explicit that this is a graded decision with a long tail: Week 4 asks for a “tech stack decision” and a Draft 1, Week 7's Checkpoint 2 asks for a “working app with at least three features” and a “tech stack rationale”, and the Week 10 Digital Prototype Report must document “architecture overview, feature set, tech stack, access instructions for assessors, known limitations”. The stack we pick now is examined three separate times, and the third examination is written down for an external reader.

[CONSTRAINT] The build window is short and its shape is fixed by the calendar. Weeks 4 to 7 carry four scheduled 4-hour workshop blocks plus out-of-class build time, and the three features due at Checkpoint 2 are not three independent toys — they are three surfaces over one event stream (Live View produces the events, Review traces them, Recipe changeover governs them), which is the property the Week 2 architecture was designed to guarantee by construction. A stack that forces us to write the event model twice — once per platform — spends our entire budget on plumbing.

[DECISION] So the question this document answers is deliberately narrow, and we state it precisely because a vague question here produces a vague answer: *which combination of framework, language, data layer and delivery pipeline lets a four-person student team, with existing React/TypeScript/Python skills and no dedicated mobile engineer, put a glove-operable, camera-fed, offline-first inspection HMI in front of real users by Week 9, on Android and iOS handsets and panels plus desktop and mobile browsers, without throwing any of the Week 2 design work away?*

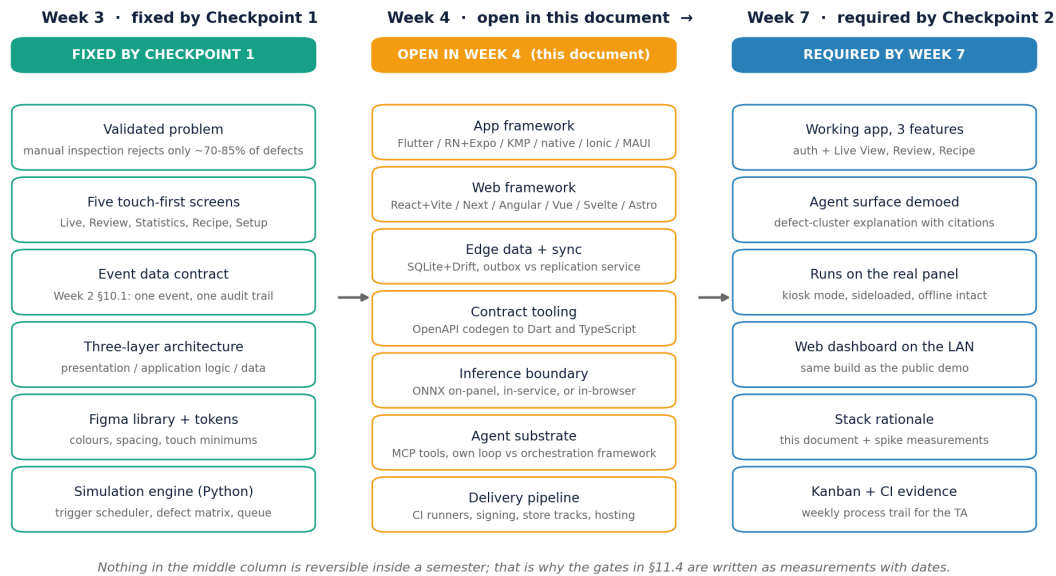


Figure 1 — The Week 3 to Week 7 crossing. Left: what Checkpoint 1 already fixes (the validated problem, five screens, the event contract, the three-layer architecture). Right: what Week 4 must newly supply. The middle band is the decision surface this document covers; the arrows show which Week 2 artefacts each stack choice must consume rather than redraw.

2 | The four delivery targets, and what each one actually asks for

[SCOPE] The product ships to four targets. They are not four equal children, and pretending otherwise is how teams end up building four mediocre apps instead of one good one. Table 2 rates each target on what it demands from a stack, and — more usefully — on who is looking at it.

Target	Who	What it demands of the stack	Hardness
T1 Android (line-side panel, tablet, handset)	Line operator, shift leader; 7-inch panel at eye height, gloves, glare, ambient noise	Sustained 60 fps painting of camera frames plus overlay boxes and a PASS/FAIL banner; large touch targets with no hover state; kiosk/immersive mode and sideload without a store account; camera and file access under scoped permissions; survival of a mid-shift reboot	Highest — this is the product
T2 iOS (handset, tablet)	Quality engineer, supervisor on the move; also the assessors' own devices	Same surfaces, different platform: push, camera permission flow, background refresh limits, and a signing pipeline that a four-student team can actually operate	High, but T1-gated: nothing is designed for iOS that is not already true on Android
T3 Desktop web	Quality engineer (deep Review, Statistics, export), and Demo Day projection	Data-dense tables with virtualisation, filters and charts at 1080p+; keyboard-first interaction; multi-monitor; no install step for a plant PC	Medium — the strongest ecosystem of the four
T4 Mobile web	Supervisor walking the line; the link we send people during promotion (Week 8)	The same React code at 390 px; installable PWA shell; graceful degradation where the browser has no camera or no background sync	Medium, and it is the promotion surface

Table 2 — Delivery targets ranked by how much they constrain the stack. T1 and T2 share one codebase under the chosen stack; T3 and T4 share the other.

[OBSERVATION] Two of the four targets are free. T3 and T4 are the same codebase if the web front end is built responsively, which every serious web framework now assumes. T1 and T2 are the same codebase under any cross-platform framework, and are two full codebases under a native strategy. So the real question is never “four targets” — it is “how many codebases”, and the honest count for the options we considered is: two (cross-platform app framework plus web), three (native Android, native iOS, web), or one (web-only, wrapped). Section 7 scores those.

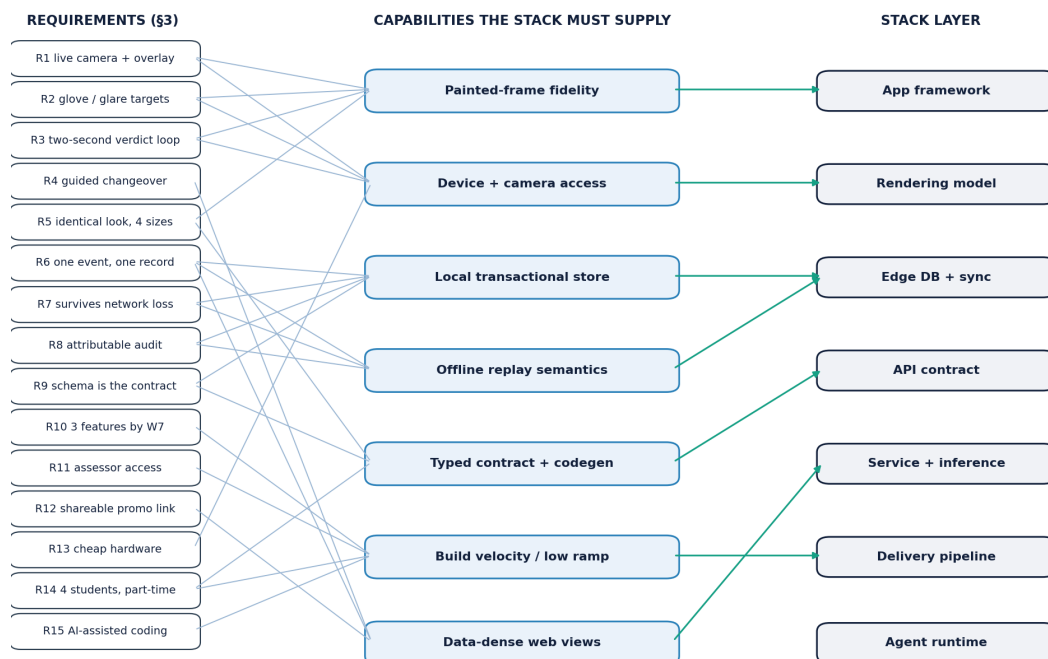
3 | Requirements the stack must satisfy

[REQ] Requirements were lifted from the Week 2 deliverable rather than invented for this document, which matters for the audit trail: R1–R5 come from §9 (the five screens and their glance/glove/glare design rules), R6–R8 from §10 (the event, audit and recipe data logic), R9 from §11 (the three-layer architecture and the toolchain table), R10–R12 from the module handbook's Week 4–7 deliverables, and R13–R15 from our own operating constraints as a student team with no budget and no IT department on the plant side.

ID	Requirement	Stack-visible?
R1	Live View renders a camera frame, a defect bounding box and a verdict banner on one surface, and stays fluid through hundreds of cycles per shift	Yes — rendering model, not a feature flag
R2	Every interactive target is operable with gloved fingers and readable under overhead glare; contrast and hit area are design constraints, not preferences	Yes — widget library and theming control
R3	Verdict-to-acknowledgement latency stays inside the two-second window in which containment is still possible	Yes — frame decode, overlay, and write-path latency
R4	Recipe selection at changeover is guided, confirmable, and versioned, with the wrong recipe made hard to select	Yes — flow authoring and local state
R5	The five screens behave identically in look and feel on a 7-inch panel and a 6-inch handset	Yes — layout and scaling model
R6	Every inspection produces exactly one event record, written locally first, with identity, timestamp, recipe version and sync status	Yes — local database and transaction model
R7	The station keeps recording through connectivity loss and reconciles without duplicates when the link returns	Yes — outbox/queue and idempotency support
R8	Audit and reclassification actions are attributable to a badge-tapped individual and are not editable after the fact	Yes — auth model and append-only storage
R9	The Week 2 event schema (§10.1) is the contract; the build phase implements it rather than redesigning it	Yes — schema-first tooling, codegen
R10	Three working features by Checkpoint 2 (Week 7), including authentication and one AI-agent surface introduced in the Week 4 agent workshop	Yes — build velocity and boilerplate cost
R11	An access route for assessors that does not require them to own an Android developer account	Yes — web link, store or QR sideload
R12	Promotion in Week 8 needs a shareable link and a landing surface that a browser opens instantly	Yes — web deployment
R13	Runs on hardware we can buy or borrow: a mid-range Android panel, an iPhone for parity, a laptop, a free cloud tier	Yes — minimum spec and licence cost
R14	A four-person student team with React/TypeScript/Python experience, no dedicated mobile engineer, and roughly 6–9 build hours per person per week outside class	Yes — ramp time is the dominant cost
R15	AI-assisted development throughout, so the framework's public code corpus and documentation quality are a real input, not a nice-to-have	Yes — model familiarity of the language and stack

Table 3 — Requirements register. The right-hand column marks the requirements that a stack choice can break on its own; these are promoted into the scoring criteria of §5.

[MAPPING] Figure 2 shows the mapping we actually used: each requirement to the capability it demands, and each capability to the layer of the stack that has to provide it. Three capabilities carry most of the decision weight — painted-frame fidelity (R1, R3, R5), local-first persistence (R6–R9), and time-to-feature (R10–R15) — and they are the three that separate the candidates in §11.



Three capabilities carry most of the decision: painted-frame fidelity, local transactional store, build velocity. They are the axes where the candidates separate in §11.

Figure 2 — Requirements to capabilities to stack layers. A candidate is rejected not because it lacks a feature but because it cannot supply a capability the product's core interaction depends on.

4 | Constraints inherited from the prototype phase

[CONSTRAINT] Four things were already decided before this workshop, and the stack has to live with them. **(i) The event model is fixed.** Week 2 §10.1 defines the inspection event, its identity, timestamp, recipe version, verdict, defect class, confidence, acknowledgement and sync status; the audit trail and the traceability promise are computed from that one table. Any stack that cannot express a local, transactional, append-only store is out regardless of how pretty its widgets are. **(ii) The three-layer architecture is fixed.** Presentation, application logic, and data/simulation, with narrow interfaces between them — which is what allows the simulated inference stub to be swapped for a real model without touching the screens. **(iii) The simulation machinery ships.** The Week 2 toolchain note committed us to the principle that “the machinery that demonstrates the concept is the same machinery that ships”: the trigger scheduler, the defect-probability matrix and the offline queue are production code with a fixture plugged into them, not throwaway demo scaffolding. That principle is what keeps Python on the server side in §10. **(iv) The design already exists in Figma,** with the five screens as interactive frames; a stack that cannot reproduce a custom-drawn verdict surface without fighting the framework destroys that asset instead of consuming it.

[NOTE] One inherited choice is *deliberately* revisited here rather than honoured: the Week 2 toolchain table named “React + TypeScript on Vite” as the build-phase front end, written when the only surface in mind was a browser-runnable prototype. That remains true for T3 and T4 and is confirmed in §8.1. It was never a claim about the operator HMI on a line-side panel, and the Week 4 brief asks exactly that question. Saying so plainly is the difference between a decision record and a rationalisation.

5 | The decision instrument

[METHOD] The instrument was written before the candidates. This is not ceremony: a comparison table assembled after the team has a favourite quietly becomes a justification, and the module’s own assessment language — “tech stack rationale” at Checkpoint 2, “known limitations” in the final report — rewards the opposite. So criteria and weights were fixed in the Week 4 workshop from the requirement register in §3, scored independently by three people, and only then were the numbers allowed to speak.

5.1 Criteria and weights

ID	Criterion	What a high score means	Weight
C1	Target coverage	One codebase genuinely serves the surfaces it claims to, including the odd ones (7-inch panel, kiosk mode, mobile browser), rather than serving them in a demo video	15%
C2	Interaction and rendering fidelity	The Live View holds its frame budget while painting camera frames, boxes and a banner; custom drawing is the normal case, not the escape hatch	12%
C3	Device and camera integration	Access to camera streams, files, notifications, printers, serial/USB and device-owner mode through maintained plugins, without writing platform channels by hand	12%
C4	Local-first data and sync	A real transactional local store, typed schema, queue-with-retry, and a sane path to conflict handling	10%
C5	Team fit and ramp	The language and mental model are ones the team can be productive in within about two weeks, given what they already know from Week 2	13%
C6	Time to Checkpoint 2	Three features, auth, and a demoable build are reachable inside four weeks of part-time work, including the boring parts (signing, store tracks, devices)	13%
C7	Ecosystem depth	Data grids, charts, state management, forms, accessibility, testing harnesses exist and are maintained by someone other than us	9%
C8	Learning corpus and AI-assistability	Public code, official docs and error messages are abundant enough that AI-assisted development — our stated way of working (R15) — is fast rather than lucky	8%
C9	Longevity, governance and cost	The project survives its maintainers: an active foundation or company behind it, a licence we can ship on, and no per-seat cost we would have to hide from a customer	8%
Total			100%

Table 4 — Criteria, weights, and the anchored meaning of a high score. Weights sum to 100%; C2+C3 (24%) and C5+C6 (26%) are the two opposing blocs, and §11.3 shows what happens when either is emphasised.

[NOTE] Two weights are worth defending out loud. **C8 at 8%** is a 2026 weight, not a textbook one: our working method is AI-assisted, and a framework whose public corpus is thin produces confidently wrong code that the team is too junior to smell. We measured C8 as a combination of language rank in public code, the size and staleness of the official documentation, and how often the framework’s own idioms appear in model answers. **C5 and C6 together at 26%** encode the fact that this is a semester, not a startup runway: a technically superior stack that costs us three weeks of ramp is not superior. Section 11.3 stresses this bloc specifically because it is the one a reader may reasonably weight differently.

5.2 Scoring rubric

Each cell is scored 1–5 (halves allowed) against the anchor definitions in Table 4, using the following scale. Scores are claims, and every score below 4 or above 3 is footnoted in Appendix B with the evidence that produced it — a documentation page, an issue thread, a plugin we opened, a device we ran.

Score	Meaning
5	Removes the constraint entirely; we would choose this even with no budget pressure
4	Meets the requirement with documented, maintained support

Score	Meaning
3	Meets it with a known workaround we would have to own; the workaround is a scheduled task, not a risk
2	Achievable only by cutting a requirement, and we would be cutting one we said we valued
1	Blocks the requirement; the product would have to change shape

5.3 Who scored, and how disagreement was handled

[EVIDENCE] Three scorers working independently: the app lead (platform and device experience), the web and data lead (front-end and API), and the product/quality lead (personas, evidence and commercial framing, no mobile background — deliberately included as the non-expert view). Each scored the same sheet blind, then cells differing by more than one point were discussed once and re-scored; no averaging across unresolved disagreement. Two criteria were *not* scored by consensus: C2 and C3, which were scored by the app lead alone against a written test (the spike protocol in §13.2), because they are hardware claims and the other two scorers had not held the panel. Final totals are weighted sums of the consensus sheet; Appendix B carries all three raw sheets so a reader can re-run the arithmetic with their own weights.

[ASSUMPTION] One bias is worth naming before it is accused: the team's existing React skill makes a React-shaped answer comfortable. That is exactly why C5 and C6 were weighted *before* candidates were listed, and why the React-shaped candidate (React Native + Expo) stayed in the shortlist to the end rather than being dismissed. It lost by 0.29 points on the sheet, and §11.3 shows the specific re-weighting under which it wins — which is the honest form of this decision.

6 | The longlist

[SCOPE] Ten app-side and nine web-side candidates were surveyed. The longlist is deliberately wider than the four options a student team would normally be shown, for two reasons: the module handbook explicitly permits low-code platforms, so they must be scored rather than snubbed; and the rejected options are the part of this document a mentor is most likely to probe.

6.1 App-side longlist

	Candidate	One-line description	Status
	Flutter + Dart	Google's UI SDK: a single codebase compiled ahead-of-time to a self-painting canvas on Android, iOS, web and desktop	Shortlist
	React Native + Expo	Meta's React-native-runtime approach, with the Expo application framework, EAS build service and managed workflow	Shortlist
	Kotlin Multiplatform + Compose Multiplatform	JetBrains' shared-logic approach, optionally extended to shared UI, targeting Android and iOS from one Kotlin codebase	Shortlist
	Native pair: Kotlin/Jetpack Compose + Swift/SwiftUI	Two first-party toolkits, one per platform, at the price of two of everything	Shortlist
	Ionic + Capacitor	Web app in a native shell: Angular/React/Vue UI distributed by the platform's WebView, with a plugin bridge to device APIs	Shortlist reserve
	.NET MAUI	Microsoft's cross-platform XAML toolkit, successor to Xamarin.Forms, C#/.NET to Android and iOS	Eliminated (§9.1)
	NativeScript	Direct binding to native widgets from TypeScript/Angular, owned by Progress	Eliminated (§9.2)
	PWA only (no native shell)	One web codebase, installable, no stores — the single-codebase extreme	Eliminated (§9.3)
	Low/no-code platforms (Softr, Lovable, Base44)	The option the handbook names first; visual builders over a hosted data layer	Eliminated (§9.4)
	Electron / Tauri desktop shell	Desktop-first packaging of a web app; considered only as a route to the panel	Eliminated (§9.5)

Table 5 — App-side longlist. Marks are the projects' own logos, supplied under Simple Icons (CC0) and Devicons (MIT); see Appendix C for attribution and licence terms.

6.2 Web-side longlist

	Candidate	One-line description	Status
	React + TypeScript + Vite	Client-rendered SPA, no server runtime, static files served from anywhere	Shortlist
	Next.js	React's meta-framework: SSR, route-level data fetching, server components, image and edge services	Shortlist

	Candidate	One-line description	Status
	Angular	Google's batteries-included framework: DI, router, forms, HTTP client, signals-based reactivity	Shortlist
	Vue 3 + Nuxt	Evan You's progressively-adopted framework and its full-stack companion	Shortlist
	Svelte 5 + SvelteKit	Compiler-first: components become imperative JS with no virtual DOM	Reserve
	Astro	Content-first architecture shipping zero JavaScript by default, with islands of interactivity	Reserve
	SolidJS + SolidStart	Fine-grained reactive signals with real DOM nodes; small and fast	Eliminated (\$9.6)
	Qwik + QwikCity	Resumability instead of hydration; near-zero startup cost	Eliminated (\$9.6)
	Remix / React Router 7	Web-standards-first React routing model, now merged into React Router	Eliminated (\$9.6)

Table 6 — Web-side longlist. The four reserves were scored but not profiled to the same depth, because each failed an earlier gate (team skill overlap or component ecosystem) than the shortlist did.

[NOTE] Absent from the web longlist, and deliberately: **Blazor** (C# in the browser — no path to the team's existing components or ecosystem), **HTMX/Lit/Alpine** (excellent tools, wrong shape for a data-dense multi-screen application surface), **jQuery-era server-rendered templates** (the Review and Statistics screens are interactive grids, not documents), and **writing the web front end in Flutter** (scored in \$11 as part of the Flutter option, and rejected on the same grounds as \$9.5: Flutter's web output is a canvas, which is the wrong substance for a table of 20,000 filterable inspection events).

7 | App-side shortlist: profiles

Each profile below answers the same five questions in the same order: what the thing is, where it came from, what it is genuinely good at, where it costs us, and what it would mean for LineSight. The origin paragraphs are not decoration. A framework's history predicts its behaviour: Flutter paints its own pixels because it grew out of a browser-engine project; React Native's rendering path is a bridge because React's problem in 2013 was Facebook's iOS app, not a canvas; MAUI's awkwardness is the residue of a 2016 acquisition. Knowing which decisions a tool already made for you is how a four-person team avoids discovering them in Week 6.

7.1 Flutter

Flutter	language: Dart	licence: BSD-3-Clause	governance: Google, with the Dart language under ECMA-408
	First release: Flutter 1.0, December 2018 (public preview November 2017). Lineage: Dart presented October 2011 by Lars Bak and Kasper Lund at Google; Sky, the rendering prototype on which Flutter was built, demonstrated 2014; Flutter 1.0 December 2018; 2.0 (web, desktop) March 2021; 3.0 (stability promise) May 2021; Impeller rendering engine default on iOS from 3.10 (2023). Current: 3.47.0 stable as of October 2026, on a roughly quarterly cadence; the exact baseline we pin is in Table 10 (\$10.8).		

Where it came from

Flutter's ancestry runs through Chromium, not through Java. In 2012 Google's mobile team was carrying a research prototype called Sky: an app shell that skipped the platform widget stack entirely and painted itself, borrowing the compositor that made Chrome fast. The version that shipped in 2014 was Android-only and drew on Skia — the 2-D graphics library Google had acquired in 2005 and since used inside Chrome and Android itself. Dart, the language, had been announced three years earlier by V8 author Lars Bak as a way to give large web teams a structured, ahead-of-time-compiled language without leaving the browser's performance envelope; it was initially received badly (it shipped its own standard library and appeared to compete with JavaScript) and was quietly rebuilt around an incremental compiler and a strong static type system. When Flutter was announced in 2015 and released in 2017–18, the two threads were welded together: Dart as the language whose AOT compiler and garbage collector were tuned for smooth animation, Skia — now increasingly Impeller, a rendering engine designed for the mobile frame budget — as the painter. That is why Flutter has no bridge, no WebView, and no dependency on the platform's UI kit: it is a browser engine minus the web, pointed at your app.

What it is genuinely good at

- **Frame-time predictability.** Everything the operator sees is drawn by our code into a surface, so a PASS/FAIL banner, a bounding box tracking a moving part, and a heat-map overlay are the same kind of object — a CustomPainter in a

widget tree that repaints at 60 or 120 Hz. The jank that historically plagued cross-platform UI (bridge marshalling on the raster thread) does not exist in this design. This is the single criterion on which LineSight's core interaction is judged (R1, R3, C2).

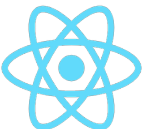
- **One codebase, four renderers, consistent pixels.** The same Dart source produces Android and iOS binaries and, if we ever want it, a canvas-based web build. Because Flutter ships its own widgets rather than wrapping platform ones, the 7-inch panel and a colleague's iPhone show the same layout — which is exactly what requirement R5 asks for, and exactly what a WebView-wrapped app gets wrong on a \$120 panel with an old Android.
- **A device and camera plugin surface that is maintained by people other than us.** Camera streaming with a low-latency preview, image capture, file and path providers, permissions, shared preferences, serial/USB bridges, BLE, background tasks, and Google's ML Kit bindings all exist as first-party or widely-used packages (C3, §10.4). The one gap we name rather than discover later: Microsoft ships no first-party Dart binding. ONNX Runtime is reached either through `flutter_onnxruntime` (1.9.0, October 2026, which bundles ORT 1.28 rather than the current 1.30) or by writing our own Dart FFI against the C API. Both routes are on the Week 4 spike list, and the pass gate is written in §13.2.
- **Local data as a first-class citizen.** `sqflite` and `Drift` give a typed, async, reactive layer over SQLite, which means the Week 2 §10.1 event schema lands almost unchanged, with real transactions and a queue we can reason about (C4, R6–R7).
- **Developer experience under a deadline.** Hot reload preserves widget state, the DevTools performance and memory views are unusually good, and `flutter create` produces a buildable project for both stores in one command — a non-trivial share of C6.
- **A public corpus that models know — smaller than React's, large enough.** Dart draws 5.9% of Stack Overflow's 2025 technology respondents against Kotlin's 10.8% and Swift's 5.4%, so this is the one place where the honest score is 4/5 rather than 5/5: model fluency in Flutter is good and improving, and the documentation site is unusually well indexed, but the answer density per question is a fraction of React's (C8, R15).

Where it costs us

- **Dart is a new language for every member of the team.** It is a friendly language — Java-shaped syntax, sound null safety, an analyzer that explains itself — but “friendly and unfamiliar” still costs the first two weeks of productivity, and our React/TypeScript experience does not transfer as directly as it would to React Native (C5, R14). This is the real price of the decision and §11.3 tests whether it is affordable.
- **Platform-native feel is ours to build.** Because nothing maps to Cupertino or Material widgets automatically, an iOS screen that behaves like an iOS screen is written, not inherited. LineSight is a purpose-built HMI, not a settings page, so we pay this cost knowingly — but it is a cost.
- **Binary size and first-launch cost.** An AOT Flutter APK is larger than a minimal native one; on the panel it is a rounding error, on a low-end handset it is a download decision.
- **Web output is the wrong substance for our web surfaces.** Flutter web compiles to canvas rendering (with an HTML renderer in maintenance posture), which is excellent for a kiosk display and poor for a filterable 20,000-row event grid with selectable text and keyboard navigation. We therefore do not use Flutter on T3/T4 — see §9.5.
- **Bus factor inside the team.** If the one person who learned Dart is unavailable in Week 9, nobody else can ship a fix. Mitigated by the pairing rule in §13.1 and by keeping the business logic in the server where all of us are fluent.

[VERDICT] Adopted for T1 and T2. Flutter tops the app-side sheet at 4.14 weighted points (§11.1) — but only 0.055 clear of React Native, so the sheet is not the argument. The argument is that Flutter's lead sits entirely on the criteria §11.4 turns into gates (C2 rendering fidelity, C4 local-first data, C1 target coverage), while its deficit sits on one criterion (C5 team fit) that we buy back on the web side, where the same React skill is spent instead of set aside.

7.2 React Native with Expo

React Native + Expo language: TypeScript licence: MIT (RN), MIT (Expo) governance: Meta plus the React community	
	First release: React Native announced February 2015, open-sourced March 2015; React itself by Jordan Walke at Facebook, 2013. Lineage: React (2013) → React Native (2015) → Hermes VM (2019) → New Architecture / Fabric announced 2022, default from 0.76 (2024) → React Compiler. Expo: open-source toolchain from 2015, EAS Build from 2021. Status: the strongest claim on the sheet for skill reuse; see §11.3 for what it would take to win.

Where it came from

React Native is a side-effect of a different problem. In 2011 Jordan Walke was working on XHP, a server-side component system in PHP, and asked whether it could produce HTML on the client with real DOM nodes as the target; the answer was React, announced in 2013 with the virtual DOM as its public face. React Native came two years later out of a realisation

inside Meta's own teams — the same declarative component model could target *native* views instead of DOM nodes. The result is a design that is genuinely clever and genuinely constrained: JavaScript describes a tree, and a bridge (later JSI, a C++ interface that removes the JSON serialisation hop) tells platform views what to do. The consequence of that ancestry is architectural and permanent: the thing on screen is a `UIView` or an `android.view`, not a pixel we chose. Anything the platform widget set cannot express — a bounding box tracking a moving part at 60 fps, a custom-painted verdict banner — needs either the canvas/Gesture Handler escape hatches or a native module written by us. Expo's role in the same history is as the developer-experience layer: it took the parts of mobile builds that eat student weeks — provisioning, signing, native dependency drift, store submission — and turned them into a service plus a managed project format, to the point where Meta's own documentation now recommends a framework-level tool as the default way to start.

What it is genuinely good at

- **Skill reuse, which for us is the highest-scoring single cell on the entire sheet.** The team already writes React and TypeScript; R14's ramp cost is close to zero, and the same mental model — components, hooks, declarative state — serves the app and the web front ends. C5: 5/5.
- **The camera and device plugin ecosystem is excellent, and in one respect better than Flutter's.** VisionCamera (frame-processing at near-native throughput), ImagePicker, MMKV, react-native-unistyles, react-native-reanimated for 60 fps animation off the JS thread, and maintained ONNX and LiteRT bridges. C3: 4.5/5.
- **Expo removes the boring weeks.** Managed builds, over-the-air updates outside the review queue, EAS Build so we do not need a Mac for every iOS attempt, dev-client for custom native code, and a preview app that gets a build onto a phone in minutes. C6: 4.5/5.
- **Web as a second-class but real citizen.** react-native-web lets shared components render in a browser, which is how teams like ours end up with a tablet layout nearly for free — though not a data-dense desktop one.
- **The largest AI-legible corpus of any mobile stack.** React is used by 44.7% of Stack Overflow's 2025 technology respondents — more than twice Angular's 18.2% and Next.js's 20.8% — and TypeScript, the language this path is written in, became the single most-used language on GitHub in Octoverse 2025. Model fluency in RN idioms is high, and the errors we will hit are already answered thousands of times. C8: 5/5.

Where it costs us

- **The rendering model is the wrong shape for Live View.** Fabric and the New Architecture narrowed the gap, and Reanimated keeps animation off the JS thread, but the fundamental contract is still “describe native views”. A surface that repaints a camera frame plus overlays hundreds of times a shift, under a two-second human loop, is the case where the indirection shows. This is exactly what the Week 4 camera spike in §13.2 measures, and it is the criterion (C2: 3.0/5 on the consensus sheet) that decides the gate.
- **Version coupling across three ecosystems.** React, React Native, and Expo each move on their own cadence, and the intersection of compatible versions is where student projects lose days. The library-native-module boundary is smoother than it was, but it is still a boundary.
- **Local-first data needs a third party.** There is no equivalent of Drift's typed, reactive SQLite layer that is maintained inside the ecosystem; teams assemble SQLite drivers plus a sync engine, or move to WatermelonDB or RxDB and accept their constraints (C4: 3.0/5). We note the 2026 state of both honestly: WatermelonDB's documentation changelog stops at 0.27.1 (October 2023) with open bridgeless-mode issues, and RxDB 17 (March 2026) moved its free tier to a 13-collection cap with paid premium features.
- **Upgrade churn as a cultural property.** RN's history is a sequence of reinventions — bridge to JSI, class to hooks, legacy to New Architecture, and the long tail of community libraries that stalled mid-transition. Each migration was an improvement; the accumulation is a maintenance tax we would be paying in Week 6.

[VERDICT] Not adopted; retained as the named fallback. React Native is the candidate this decision could most plausibly have gone the other way on, and we say so in §11.3: weighting team fit and time-to-checkpoint at the expense of rendering fidelity puts it first. It loses on the surface that defines the product. It is also the documented retreat path if the Flutter camera spike fails its gate — the migration is expensive but bounded, and the team's skills survive it intact.

7.3 Kotlin Multiplatform with Compose Multiplatform

Kotlin Multiplatform language: Kotlin licence: Apache-2.0 governance: JetBrains with the Kotlin Foundation	
	First release: Kotlin announced February 2011 by Andrey Breslav's team at JetBrains; 1.0 February 2016; designated Google's preferred Android language at I/O 2019; KMP for iOS production use from Kotlin 1.9.20 (November 2023); Compose Multiplatform desktop stable 2023, iOS stable 2024. Status: the most credible “shared logic, native UI” middle path, and the one whose evidence base is thinnest for a team of our size.

Where it came from

Kotlin was a language project before it was a mobile project: JetBrains built it from 2010 for a problem their own IDEs had — Java's verbosity, and the inability to evolve a language inside a platform whose governance moved slowly. Its pragmatic design (interoperability with Java as a hard constraint, null safety, extension functions, coroutines borrowed from CSP research) is what made Google's 2019 endorsement easy. Multiplatform came from the same instinct: compile the same Kotlin to JVM, JavaScript and native code, so that business logic stops being written twice. Google's K2 compiler collaboration and JetBrains' investment pushed the iOS target to officially supported status in late 2023. Compose Multiplatform extends the Android UI toolkit — itself announced at I/O 2019 and 1.0 in July 2021, a declarative layer over a canvas-style renderer that Android contributed to Skia — to desktop and then iOS, which means the same Kotlin code can draw a screen on both platforms without a widget bridge. Conceptually, Compose Multiplatform is closer to Flutter's model than to React Native's.

What it is genuinely good at

- **Incremental adoption with no cliff.** KMP's killer property is that you can share the domain layer, the event model, the sync queue and the API client, and keep Android and iOS UIs native. For a product that must feel right on both handsets, that is an attractive shape (C1: 3/5 overall, because the UI is still two codebases).
- **Coroutines are the best concurrency story of any candidate** — structured concurrency, cancellation scopes, flows — and they map beautifully onto LineSight's event stream and offline queue (C4: 4.5/5 with SQLDelight).
- **SQLDelight and Ktor are mature, typed, and compile-time-checked** — a schema-first story that fits R9 better than anything except codegen-over-OpenAPI.
- **Android-first performance and platform access are unimproved, because they are native.** Camera, device-owner mode, ML Kit, NNAPI: nothing is behind a plugin author's weekend.

Where it costs us

- **Two UI codebases, or one young one.** Either we write Compose for Android and SwiftUI for iOS — which is the native-pair cost in §7.4 with a shared service layer bolted on — or we adopt Compose Multiplatform on iOS, whose production track record outside large companies is still short, and whose iOS-specific behaviours (text input, keyboard, accessibility, scroll physics) are the parts a student team is least equipped to debug.
- **Nobody on the team writes Kotlin today, and iOS is the harder half.** Even the Android-side learning is a full language plus a declarative toolkit; the iOS side additionally requires Xcode fluency. C5: 2.5/5 — the second-lowest score any candidate received on any criterion.
- **No path to T3/T4.** The web surfaces stay a separate project regardless, so the codebase count is three (Android UI, iOS UI, web) — the property that makes this option weak on C1 despite its excellent shared-logic story.

[VERDICT] Not adopted. The best architecture on paper for a company with an Android-fluent team and a long runway; the wrong shape for four students with four weeks and a React skill base. If LineSight survives the module and hires an Android engineer, this is the documented migration path for the shared domain layer — and the reason we keep our business rules out of Flutter widgets in §12.2.

7.4 The native pair: Kotlin/Jetpack Compose and Swift/SwiftUI

Native languages: Kotlin, Swift licence: Apache-2.0 / Apple toolchain governance: Google and JetBrains / Apple			
			
First release: Android 1.0 with Java in 2008 (Google acquired Android Inc. in 2005); Jetpack Compose 1.0 July 2021. Swift announced 2014 by Chris Lattner's team, open-sourced 2015; SwiftUI announced 2019. Status: the ceiling every other option is measured against, and a budget line we cannot fund.			

Where it came from

Android was built in Java on a purpose-made VM (Dalvik, later ART) because Google needed a mobile platform that the existing Java ecosystem — and a workforce — could move into without retraining; the ten-year litigation over that single choice is itself a lesson about how expensive a stack decision is. Jetpack Compose arrived when Google finally accepted that the view hierarchy had aged, and took the React insight — declarative UI as a function of state — and implemented it as a compiler plugin that emits its own render tree. iOS's story is the mirror image: Objective-C from NeXT in 1988, inherited wholesale in 1997 when Apple bought NeXT, then Swift from 2014 as Apple's attempt to give its developers a modern language with Lattner's compiler pedigree (LLVM) behind it, and SwiftUI in 2019 as Apple's declarative answer, built on the same Combine/state-observation ideas.

What it is genuinely good at

- **Maximum on every capability the platform offers.** Camera pipelines, device-owner and kiosk modes, background execution rules, Core ML and the Apple silicon neural engine, NNAPI and GPU delegates, accessibility, and platform-correct input behaviour. Both C2 and C3 score 5/5, and no cross-platform option beats them — the question is whether the gap matters for our surfaces.

- **Zero abstraction tax when a framework updates.** A new OS behaviour is available on day one, not when a plugin maintainer agrees.
- **Longest survivability horizon.** These toolkits outlive every product built on them. C9: 5/5.

Where it costs us

- **Two codebases plus the web front end is three, and the event model is written and tested three times.** R6–R9 — one event, one audit trail, one schema — is precisely the requirement that punishes duplication: every sync-status rule, every idempotency guard, every recipe-version check would exist in Kotlin, in Swift, and in TypeScript.
- **Nobody on the team has shipped an iOS app, and the signing/provisioning layer is a project in itself.** C5: 2/5 and C6: 2/5 — the lowest time-to-checkpoint score on the sheet.
- **Design drift.** Two teams of students drawing the same five screens twice produces two products, and our personas need one station.

[VERDICT] Not adopted. This is the option we would choose with a mobile engineer on staff and a two-year roadmap. With neither, it spends our entire schedule on plumbing and buys fidelity we cannot spend. It remains the reference point for “what we are giving up” in §11.4 — and the honest answer is: about 15% of the frame budget, in exchange for one codebase and four weeks.

7.5 Ionic with Capacitor

Ionic + Capacitor language: TypeScript licence: MIT governance: Ionic (private company)	
	First release: Ionic 2013 (Max Lynch, Dan Imhoff, Adam Bradley), built on Angular and Cordova; Capacitor — Ionic’s own native runtime — announced 2019; Ionic React and Vue support 2019. Status: the maximum-reuse option, and the minimum-fidelity one.

Where it came from

Ionic is the direct descendant of PhoneGap/Cordova, which in 2009 made the observation that a mobile browser was already a UI toolkit and all that was missing was a bridge to the device. That bet — web skills, web substance, native access through a plugin shim — defined the category. Capacitor exists because Cordova’s plugin architecture aged badly: Ionic rewrote the shell so that native code is a first-class Swift/Kotlin project rather than a config file, and shipped a plugin set (camera, filesystem, preferences, push, background tasks) on top of it.

What it is genuinely good at

- **One codebase for all four targets — the only candidate that literally does it.** If LineSight were a form-and-list application, this would be the rational choice (C1: 4.5/5, the highest score any candidate received on that criterion).
- **Total skill reuse:** our Week 2 React/TypeScript front end would *be* the app, which is also the reason the option is worth taking seriously rather than dismissing.
- **Cheap iteration and instant updates:** no rebuild for UI changes, and the same artefact runs in a browser for the assessors (R11).

Where it costs us

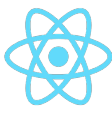
- **A WebView is the renderer.** Camera preview inside a WebView on Android is a well-known source of latency and z-order pain; continuous frame delivery to JavaScript crosses the bridge on every frame. C2: 2.0/5 and C3: 3.5/5 — the same two criteria Flutter wins on, and the reason the option is scored rather than shortlisted.
- **Touch fidelity on cheap hardware.** Input latency, scroll physics and keyboard behaviour on a \$120 panel are exactly where WebView apps earn their reputation, and R2 (gloves, glare, two-second loop) is unforgiving here.
- **Local-first data is browser storage.** IndexedDB plus a sync layer is workable; it is not a transactional event store with the guarantees R6–R8 imply.

[VERDICT] Not adopted. The reuse argument is genuinely strong for a CRUD-shaped product; LineSight’s core surface is a painted, camera-fed, timing-critical HMI, and Ionic’s renderer is the wrong substance for it. Its existence is, however, the reason we had to defend *not* using Flutter web for the browser surfaces (§9.5) — the same objection cuts both ways, and §12.4 records why we resolved it asymmetrically.

8 | Web-side shortlist: profiles

The web decision is smaller than the app decision, but it is not trivial, and it is the one an assessor is most likely to have opinions about — because it is the surface they will open on their own laptop at Demo Day. The requirement set is different too: no camera-critical painting, but instead dense filterable tables, keyboard operation, multi-monitor layouts, an instant link for Week 8 promotion, and a build that a plant IT department will allow on an internal machine.

8.1 React 19 + TypeScript + Vite (single-page application)

React + Vite licence: MIT governance: Meta + React Foundation; Vite under the VoidZero project	
  	First release: React 2013; TypeScript announced October 2012, 1.0 April 2014; Vite 1.0 2020. Adopted for T3 and T4.

Where the three pieces came from

React began as a question Lars Bak's colleague-adjacent world had already answered: if the DOM is a retained-mode graphics API that programmers manipulate by hand, what happens if you treat it as a output of a pure function of state? Jordan Walke, working at Facebook on XHP — a component system in PHP — built the first prototype in 2011, and React shipped publicly at JSConf US in 2013 with its virtual DOM and its controversial JSX syntax. The ecosystem that grew around it (Redux in 2015, hooks in 2019, the React Compiler line) is the reason a data-dense table with 20,000 rows has twenty credible component libraries. TypeScript is the other half of the story: Anders Hejlsberg, author of Turbo Pascal and Delphi and then a C# architect at Microsoft, started a language superset project in 2012 explicitly to make large JavaScript codebases maintainable — structural typing, generics, and an inference engine designed around incremental checking. It became the default way to write front-end code, and by 2025 the compiler itself was being rewritten in Go for an order-of-magnitude speedup, which matters to us because type-checking is where a codegen-from-OpenAPI workflow earns its keep. Vite is Evan You's second framework-adjacent invention: in 2020 he took the observation that a dev server does not need to bundle at all — native ES modules plus an esbuild-powered dependency pre-bundle gives instant startup — and built the tool that quietly replaced webpack across the ecosystem, including inside Vue, Svelte and increasingly React itself.

Why this combination, in four sentences

It is the stack the Week 2 prototype already assumed, so the five screens' information architecture and the §10.1 data contract carry over directly rather than being re-derived. It is the stack the team can operate on day one, which is the single largest C5 score available to any candidate. Its component ecosystem for the surfaces we actually need — virtualised grids, filter bars, date ranges, charting, CSV export — is the deepest of any framework here. And a static SPA bundle has an unusual property in an industrial setting: it can be served from a box on the plant LAN with no runtime, no server-side process, and nothing for IT to approve beyond a port, which is exactly the deployment story §12.5 needs.

What we give up, and why it is acceptable

No server rendering, so first paint on a cold 3G handset costs a bundle download — mitigated by code-splitting per route, a small dependency budget, and a service worker that makes the second visit instant (T4 is used by a supervisor on plant Wi-Fi, not by anonymous traffic). No SEO, which for an internal HMI is a non-issue; the Week 8 marketing landing page is a separate static artefact where SEO does matter, and it can be added to the same repository without changing the product build. No server components or route-level data fetching — which is a deliberate gain in predictability: our data arrives over a WebSocket and a polling query, not a request-time render.

[VERDICT] **Adopted for T3 and T4, with a PWA shell** (web manifest, service worker, offline app-shell, installable on Android and iOS Safari) so the mobile-browser target becomes an install target for promotion without a second codebase.

8.2 Next.js

Next.js licence: MIT governance: Vercel	
 	First release: October 2016 by Guillermo Rauch's team at what became Vercel; lost the web-side decision by a small margin, on operational grounds.

Where it came from

Next.js is the oldest surviving answer to "React on the server first": Rauch built it in 2015–16 as a zero-configuration server-rendering layer for React, and it grew — through dynamic imports, API routes, static export, incremental static regeneration, the app router in 2022 and React Server Components in 2023 — into a full application framework whose default mental model is "a server renders your components". Its commercial backing (Vercel) is also its constraint: the framework is genuinely open source and its features are sequenced to make the hosted platform valuable.

Why it lost

Not because it is weak — for a public-facing product it is the strongest option available — but because the properties that make it strong are the ones our deployment cannot use. Server-side rendering buys first-paint and SEO for anonymous traffic; LineSight's users are four named personas behind a login on a LAN, and the traffic is authenticated and repeat. A Node runtime on the server adds a process to operate, a version to keep current, and a thing to explain to plant IT, in exchange for nothing we asked for. Server Components introduce a boundary our team would have to reason about while also learning Dart — the

worst possible week for both. And the honest observation from the Week 8 side: the one surface where Next.js genuinely helps, a marketing landing page with SEO, can be a static file.

[VERDICT] Not adopted for the product; revisited for the Week 8 landing page only if the promo pack needs SEO, in which case it is a separate small deployment, not a change to the product front end.

8.3 Angular, and Vue with Nuxt

Candidate	Origin and character	Why not us
 Angular	Miško Hevery's 2010 Bower-side "HTML in JavaScript" project became AngularJS (2012), was rewritten in TypeScript as Angular 2 (2016) under a Google-maintained, battery-included model — DI, router, forms, HTTP, RxStreams — and has spent 2023–2026 replacing its zone-based change detection with signals and standalone components. The most enterprise-shaped of the three.	Nobody on the team writes it; its opinionated core is a poor fit for a team that wants to compose its own state layer; and RxJS is a second reactive system to learn in the same month as Dart. Scored 3.60 on the web sheet — respectable, and irrelevant against a 4.88 we did not have to earn.
 Vue 3 + Nuxt	Evan You built Vue in 2014 as a gentle on-ramp to reactive components, funded independently; Vue 3 (2020) moved to a composition API and a compiler rewrite, and Nuxt (from 2016, Sébastien Chopin) is its full-stack companion. The friendliest learning curve in the category.	Its strengths are the ones we do not need most (approachability for new contributors) and its component ecosystem for data-dense grids and industrial dashboards is thinner than React's; the team's existing React knowledge is a sunk asset we would be writing off. Scored 3.75, second of the non-React candidates behind Next.js, which is the honest measure of how good Vue is and how little of that helps us.

8.4 The reserves: Svelte, Astro, and what they taught us

Svelte (Rich Harris, 2016; reactivity-by-compiler in Svelte 3, 2019; runes in Svelte 5, 2024) and Astro (Fred Schott and Nate Moore, 2021; "islands" architecture) are both excellent pieces of engineering and both wrong for this product. Svelte's compiler approach removes the runtime we would be learning to reason about, which is a real advantage — for a team that already knows it. For us it buys a third framework in the same sprint as a third language, and its component market for virtualised tables is smaller. Astro's zero-JS-by-default premise is designed for content sites; our web surface is an application behind a login. Scoring them (3.15 and 3.08) made one finding explicit that we then used elsewhere: **on the web side, ecosystem depth for data-dense UI is worth more than runtime elegance**, and that is the criterion on which React's decade of component libraries is simply unbeatable at student scale.

9 | Options we eliminated, and the gate each one failed

Recording eliminations is the part of a decision document that earns trust at Checkpoint 2. Each entry states the gate, not the taste.

9.1 .NET MAUI — eliminated on team fit and ecosystem trajectory

.NET MAUI C# licence: MIT governance: Microsoft	
.NET	Lineage: Xamarin (Miguel de Icaza, announced 2011) bound .NET to mobile; Xamarin.Forms (2016) added cross-platform XAML; Microsoft acquired Xamarin (2016); .NET MAUI — Multi-platform App UI — shipped with .NET 6 in 2022 as the successor, unifying Xamarin.Forms, Windows App SDK and Blazor Hybrid under one C#/.NET model.

Gate failed: C5/C6 (team fit, time). Nobody on the team writes C#, and the .NET mobile toolchain's momentum in 2025–26 is visibly behind Flutter's and React Native's: fewer answers to search for, a thinner third-party component market for exactly the touch-HMI patterns we need, and a rendering model (platform-control abstraction) that inherits the same weaknesses as Ionic's without Ionic's skill reuse. On paper MAUI is a credible enterprise choice; in a semester it is a learning project with a signing problem attached. Weighted score 2.73 — last of ten on Table 7, and last of the frameworks that could genuinely ship to both handsets.

9.2 NativeScript — eliminated on maintenance signal

Progress's NativeScript (2014) shares Ionic's premise — real native widgets, driven from TypeScript — but without Ionic's distribution advantage. The project's release cadence and community size have both contracted, and a small team choosing a mobile framework is choosing the maintenance intentions of strangers. C7 and C9 both scored 2. We carried it to the full sheet anyway (2.78, eighth of ten) rather than dropping it at the longlist, because a candidate eliminated by assertion is exactly

the kind of thing a mentor asks about — and the sheet’s answer is the same as the early read: it is a competent framework whose maintenance signal, not whose capability, is the problem.

9.3 PWA only, with no native shell — eliminated on device access

PWA term coined 2015 by Frances Berriman and Alex Russell no licence, a set of web standards	
	Lineage: service workers (2014–15), web app manifests, Workbox (2017) and Chrome’s install prompt turned “web app” into something installable and offline-capable; the Web App Manifest and Background Sync specifications made it standard rather than Chrome-specific.

Gate failed: C3 (device integration), and R1/R3 on iOS. A PWA on Android can be installed in kiosk-ish mode and can hold an IndexedDB-backed queue; a PWA on iOS gets a stricter storage eviction policy, no device-owner mode, restricted background execution, and a camera path that is not a continuous stream. For a station product whose whole promise is a two-second verdict on a live feed, the browser’s camera and background limits are the product’s ceiling. We keep the PWA *inside* the strategy — it is our T4 delivery mechanism (§8.1) — while rejecting it as the only one. Scored 3.52, which surprised us: it comes within 0.04 of the native pair on the sheet, and the reason it still loses is the one criterion we refuse to trade (C3), which is what §11.4’s constraint-based reading is for.

9.4 Low-code and no-code platforms — eliminated on the prototype-to-product gap

The handbook names Softr, Lovable and Base44 first, and the Week 5–7 “platform of choice” language means a low-code route is explicitly legitimate. We scored it honestly and it scored **2.77, ninth of ten**: it wins C5 and C6 (a working CRUD app in a weekend) and loses C1–C4. Three specifics decide it. (i) Live View is a painted canvas with a camera feed and per-frame overlays — not a form or a table, and not expressible in a visual builder. (ii) The offline outbox with idempotent replay (R7) needs to run *on the device*, in code, with transactional semantics. (iii) The Week 2 §10.1 schema is a contract we intend to hand to a plant MES; a platform whose data model we cannot export cleanly puts the traceability promise — the thing enterprise buyers pay for, per Week 2 §6 — inside someone else’s database. We also note the AI-assisted-code argument in our favour: with a codebase and a language server, AI help compounds; inside a visual builder, it does not.

9.5 Flutter web, and Electron/Tauri for the panel — eliminated on substance

Two mirror-image temptations. **Flutter web** would give one language across all four targets, and its output is a canvas — the same objection we raised against Ionic, pointed the other way: Review and Statistics are text-heavy, selectable, keyboard-driven grids that must be screen-reader navigable and cheap to scroll through 20,000 rows. **Electron** (GitHub’s Atom Shell, 2013; Chromium plus Node on the desktop) and **Tauri** (2019–2022; a Rust core driving the system WebView) are the right answer for a desktop tool we were not asked to build, and the wrong answer for a 7-inch Android panel that must be installed by sideloading an APK. Both eliminated at the target-coverage gate.

9.6 SolidJS, Qwik, and Remix/React Router 7 — eliminated on ecosystem, not merit

All three are technically interesting and all three failed the same way: our web surfaces need a mature grid, chart and form ecosystem (C7), and the team needs the largest possible public corpus (C8). Solid (Ryan Solid, 2018) keeps real DOM nodes with fine-grained signals and is superbly fast; Qwik (Miško Hevery and Adam Pawlik, 2020–22) replaces hydration with resumability, an idea worth reading about regardless of what we ship; Remix (Ryan Florence and Michael Jackson, 2020, acquired by Shopify 2022) was folded into React Router 7 in late 2024, which is itself a cautionary note about betting a semester on a small meta-framework. Their web-sheet totals — 2.75, 2.20 and 3.68 — put all three below React and, for Solid and Qwik, below every other framework on the list; none of them lost on elegance.

10 | The layers behind the two headline choices

[NOTE] A stack decision that stops at “Flutter and React” is a framework decision with the hard parts still ahead of it. The layers below are where student projects actually fail: the local data model, the sync path, the contract between front end and service, and the delivery pipeline that has to survive a signing certificate. Each subsection names the choice, its origin, what it buys us, and what we accepted when we took it.

10.1 Edge data: SQLite, with Drift as the typed layer

Local store SQLite (public domain) + Drift (MIT) alternative considered: PowerSync, WatermelonDB, Isar, Realm	
 	Origin: D. Richard Hipp began writing SQLite in 2000 while at General Dynamics, for aviation maintenance systems that had to keep working when nothing else did — no server process, no administrator, a single file, transactional. It is now the most widely deployed database engine on earth, embedded in every handset, browser and aircraft we know of. Drift brought Reactive-Extensions-style streams, compile-time query checking against Dart code, and a typed DAO layer over it.

Why this and not a sync service

The Week 2 §10.1 event schema is a relational design with a sync-status column, an append-only event table, a recipe-version foreign key and an audit view — it was drawn as a SQLite schema. Using the database the contract was designed against, with real transactions and a real query planner, is the shortest path from paper to product. The alternatives were scored and rejected for reasons worth recording: **Realm** (device database with MongoDB Atlas sync) is a capable product but its device-sync line has been narrowing, and adopting it means adopting a vendor's conflict model wholesale; **Isar** was a fast NoSQL option for Flutter whose maintenance stalled; **WatermelonDB** and **RxDB** are React-side and would put our most timing critical data behind a WebView; **PowerSync** and **ElectricSQL** are genuinely good Postgres-to-SQLite replication engines and we score them as the *upgrade* path in §12.4, not the Week 4 baseline — because they solve a problem (many stations, continuous replication, conflict resolution at scale) that the pilot does not have yet, and they add a dependency we cannot debug at 2 a.m. before Checkpoint 2.

What we accepted

We own the sync logic. That is the trade: a hand-written outbox (below) is roughly 300 lines of code we control and can test, versus a service that does more and explains less. In Week 7 that is the right side of the line; in a real deployment with eight stations it is the wrong side, and §12.4 says so.

10.2 Sync: an outbox with idempotency keys, not a merge

[ARCHITECTURE] Every local write is a transaction that inserts the event row and an outbox row with a client-generated ULID, a monotonic per-station sequence, and the station's identity. A background pump drains the outbox to the central API with exponential backoff and jitter; the server answers with the accepted sequence and any authoritative fields (verdict overrides, recipe versions). Duplicates are impossible by construction because the ULID is the primary key server-side; ordering is per-station and therefore total; and a station that has been offline for a shift replays in sequence without a merge step. The Week 2 exceptions flow (`mmd_exceptions`) already specified this behaviour; the stack choice is what makes it testable.

[CONSTRAINT] Two rules keep this honest as the team grows. **(i) No clock trust.** Server time is authoritative for the audit record; device time is recorded separately as `observed_at` and `difff`, because a plant panel that reboots to 2019 is a normal event, not an edge case. **(ii) Conflicts are not merged, they are escalated.** Two humans reclassifying the same defect is a quality-engineering question with an audit answer — last-writer-wins with a visible history entry — not a CRDT problem. We note that CRDT libraries (*Automerger*, *Yjs*) exist and that the temptation to use one is real; the requirement register does not contain collaborative editing, and adding a mergeable-data-type dependency to satisfy a requirement nobody has is how offline-first projects die of sophistication.

10.3 Centre: Python + FastAPI + PostgreSQL, contract-first

Central service FastAPI (MIT), PostgreSQL (PostgreSQL licence), Docker (Apache-2.0)	
  	Origins: PostgreSQL descends from Michael Stonebreaker's Ingres/Postgres research project at Berkeley (1986), and has been extended by a volunteer community ever since — the relational system that took the enterprise features without taking a company. FastAPI is Sebastián Ramírez's 2018 re-reading of Python web services around type hints and ASGI, which made Pydantic-validated, self-documenting APIs the default in Python.

[RATIONALE] Three arguments, in descending order of weight. **Continuity:** the Week 2 simulation engine — the trigger scheduler, the per-recipe defect-probability matrix, the frame fixtures — is Python, and R9's promise is that the machinery that demonstrated the concept is the machinery that ships. **Inference adjacency:** every on-device and server-side model path in §10.4 has a Python training/export story, and putting the model behind the same service that serves the API avoids a second hop. **Contract-first:** FastAPI emits an OpenAPI document (derived from Swagger, standardised under the Linux Foundation's OpenAPI Initiative since 2015), which we then feed to two code generators — `openapi-typescript` for the React side and a Dart generator for the Flutter side — so that the event schema exists once, in Python, and both front ends are rebuilt when it changes. That single pipeline is our answer to the three-codebase problem §2 identifies.

[POSITION] NestJS (Node/TypeScript) was the serious alternative, and we rejected it on three grounds rather than a score: it is better end-to-end type sharing with the web front end (one language, one type system, trRPC-class optionality), worse for model integration (no first-class story for the export-and-run path in §10.4), and it would mean rewriting a simulation engine we have already written in a language we have not used for it. A fourth argument is unglamorous and decisive: the FastAPI/Pydantic pairing makes the OpenAPI document a by-product of code we were going to write anyway, whereas in Node the contract has to be authored twice. For a team that had not already written the engine in Python, NestJS would be the recommendation, and §14 records that as the condition under which ADR-005 would be revisited.

10.4 Inference: ONNX Runtime at the edge, the same graph on the server

[STACK] A defect-detection model is worth nothing if it cannot run where the parts are. We standardise on **ONNX** — the exchange format Microsoft, Facebook and Amazon opened in 2017 — as the boundary: train or obtain a model anywhere, export to ONNX, and run it with **ONNX Runtime** on the Android panel (CPU/GPU delegate), on the server, and in the browser with **onnxruntime-web** compiled to WebAssembly and, where available, WebGPU. The Flutter side reaches it through Dart FFI or a maintained plugin; the Python service uses the same runtime. The alternatives were **LiteRT** (Google’s rename of TensorFlow Lite), which is excellent on Android and weaker as a cross-target story, and **Core ML**, which is iOS-only and therefore a second integration path. The Week 4 spike in §13.2 measures end-to-end latency on the panel with a small detector at the resolution the station actually needs; the pass gate and the fallback (server-side inference over the LAN) are both written down before the measurement, which is the point of a spike protocol.

Why the boundary holds, and the two places it is thin

The choice is defensible because of what ONNX has become rather than what it is: as of October 2026 the runtime ships a 1.30 line on the server (10 September) and for the browser (14 September), with the web build’s monthly download count up roughly eleven-fold in a year — evidence that the format, not a vendor’s model zoo, is where the ecosystem has settled. **LiteRT** (Google’s 2024 renaming of TensorFlow Lite, current at the 2.3 line, with MediaPipe’s 1.1 task libraries on top) remains the better Android-only answer, and **Apple’s own stack** went further in 2026 with a new Core AI path and on-device Foundation Models APIs; neither is wrong, both are single-platform, and a second integration path is exactly what a four-person team cannot maintain. Two thins spots are named here rather than discovered later: **(i) the browser is not a uniform target**. WebGPU reaches roughly 88% of users but is still not an Interop/Baseline success — Firefox ships it behind a flag as we branch — while WebCodecs, which is what a live video pre-filter actually needs, is Baseline at roughly 96%. So the web inference path is written to degrade to WASM by default and use WebGPU opportunistically, and no requirement depends on **WebNN**, which remains a Candidate Recommendation Draft that no shipping browser implements. **(ii) the app-side binding is a community artefact** that bundles a slightly older runtime than the server’s, per Table 10; “one graph, three targets” is a statement about the format and an approximation about the engines. §13.2 measures the gap instead of arguing about it.

10.5 AI agents: a server-side runtime, MCP tools, swappable models

[CONTEXT] The Week 4 workshop is an AI-agents workshop, and the handbook’s Week 4 deliverable list includes an agent integration plan. We treat that as a stack question, because it is one: the choice of agent *substrate* determines what we can build in four weeks and what we will regret in ten.

Layer	Choice and reason
Protocol	Model Context Protocol (Anthropic, opened November 2024): a client/server protocol by which a model discovers tools and resources. Chosen because it turns our inspection API into a <i>tool surface</i> once, reusable by any MCP client — including the coding agents the team already uses — and because it is now the common denominator across providers: the specification is on its 2026-07-28 revision, it was donated out of a single vendor in December 2025 into a Linux Foundation umbrella, and the hosted low-code agents we scored (Dify, n8n) both moved to licences that are not Apache-2.0, which makes a protocol we can implement in a day the safer dependency. 
Model	A DeepSeek/Qwen-class API as default, with an OpenAI/Anthropic-class fallback behind one interface. Chosen on cost, latency and reachability from our network position, not on benchmark rank — the frontier-class Chinese APIs are published around \$0.30–2.00 per million input tokens and \$1.20–6.00 per million output, with million-token contexts, against materially higher Western list prices, and the pricing table moves every quarter, which is precisely why the agent layer must not care. Every surface is priced in tokens per user per session and capped in config, not in hope.   
Runtime	Python, in the FastAPI process, with a thin tool-loop we own rather than a heavy orchestration framework. Lang-Graph/LangChain were scored (they buy checkpointing and graph composition at the price of a second abstraction the team must learn in Week 5); for three tool-calling surfaces, an explicit loop with typed tools is easier to audit — and auditability is the product’s whole promise. 
Transport	Server-sent events to both front ends: streams to a Flutter Stream and to a browser EventSource without a WebSocket dependency, degrades cleanly, and is trivially observable in the network tab during user testing.
Guardrails	Every agent action is a tool call against the same permissioned API a human uses , logged with the same identity binding (R8), and no agent writes to the event store without a human confirmation step . The assistant proposes; the operator disposes.

[SCENARIO] Three agent surfaces, deliberately small, each tied to a Week 2 requirement rather than to what agents make possible. **(A) Defect-cluster explanation** (Week 7, Feature 3): “why did porosity rejects triple on station 2 since Tuesday?” — the agent queries the event store through tools, returns a cited summary, and links to the filtered Review view it derived. This is R-traceability from §2’s quality-engineer persona. **(B) Containment note drafting** (Week 8–9): from a selected set

of FAIL events, draft the non-conformance narrative and the affected serial range for a human to edit and sign. **(C) Recipe-changeover assistant** (stretch): given a work order, propose the recipe and the guided-checklist steps, with the diff from the currently loaded recipe surfaced for confirmation — R4, and the highest-consequence of the three, which is why it is stretch.

[RISK] The failure mode of agent features in student projects is a chat bubble bolted onto a product that did not need it. Each surface above names the requirement it serves and the human confirmation that bounds it; §12.6 records the risk and the mitigation (a hallucinated verdict must never reach the operator without a provenance link — the agent's output is an *annotation* on the record, never the record).

10.6 Identity, roles and the badge-tap

[STACK] **Server-side sessions with short-lived JWTs and a device-pairing step**, not a consumer identity provider. Personas are line operator (badge PIN at the station, session shared with the panel), quality engineer, shift leader, supervisor — the four from Week 2 §2. Every event carries operator_id resolved from the station session, and every audit entry carries actor_id plus reason. Why not Firebase Auth / Supabase Auth, which would be faster? Because R8's promise is that the record is defensible in a plant audit, and a plant audit asks who was at the station at 14:32 on a machine that has been offline since 14:00 — that answer has to come from our own identity binding, not from a cloud session that was unreachable at the time. We keep the option of delegating *engineer* logins to an external provider in the web tier, where connectivity is assumed.

10.7 Delivery: repositories, pipelines, and the signing reality

Concern	Choice	Why, and what it costs
Source & CI	 GitHub + Actions	One place for code, issues, Kanban evidence and pipelines; Actions' macOS runners solve the iOS build without owning a Mac rack. Cost: macOS minutes are billed at a multiplier and student budgets are not — mitigated by building iOS only on tagged release candidates.
App store pipeline	 Codemagic (or Expo EAS if we retreat to RN) for Flutter builds	Purpose-built mobile CI with signing management; the alternative is spending a week learning fastlane's match configuration. Fastlane remains the fallback for store-metadata automation.
Android distribution	Internal testing track + sideloaded AAB/APK for the panel	R11 and the plant reality: a pilot station gets an APK and a kiosk launcher, not a public listing.
iOS distribution	TestFlight via a single Apple developer account	Two weeks of lead time on provisioning is the known risk; §12.6 puts it on the register with a Week-5 date.
Web deployment	 Static build on Cloudflare Pages/Netlify for the public demo, nginx on the plant LAN box for the pilot	Same artefact, two targets; no server runtime to operate.
Service & data	 Docker Compose : FastAPI + PostgreSQL + an inference worker	One command on any machine the plant lends us; also how the assessors run it during a demo.
Versioning	Semver tags; main protected; branch per feature with PR review	Evidence for Checkpoint 2's Kanban-and-review requirement, and the only realistic way four students avoid stepping on each other.

10.8 Version baseline, verified 7 October 2026

[METHOD] "Flutter and React" is not a buildable instruction; a set of pins is. Before writing code we checked the registries and recorded what is actually current as at **7 October 2026**, so that Week 5 starts from a stated baseline instead of whatever the project template happened to ship, and so that a reader can separate the claims in this document that are about a tool's *design* — which outlast any release — from the ones that are about today's version and will need re-checking. The last block matters as much as the first: it lists what we deliberately did not adopt because the maintenance signal or the licence told us not to.

Layer	Component	Pin	What the pin means for LineSight
App — T1 panel and T2 handsets (Flutter/Dart)			
App	Flutter SDK	3.47.0 stable	Google's roughly quarterly stable cadence. We upgrade between checkpoints, never inside a feature week; the SDK is the Dart version, which removes one class of drift.
App	Drift	2.35.1 (30 Sep 2026)	Active, near-monthly. Its code generator is the part that breaks on upgrade, so generated DAO code is committed and diffable in review.
App	sqflite	2.4.4+1	Kept as the fallback store if Drift's generator fights the panel's bundled SQLite build. Two paths to the same schema file, not two schemas.
App	flutter_onnxruntime	1.9.0 (bundles ORT 1.28)	A community plugin whose bundled runtime is one minor behind the server's. §13.2's spike tests it <i>and</i> a direct Dart FFI path; risk S5 in §12.6 exists because of this row.
App	Patrol / Maestro	4.10.0 / CLI 2.11.0	Touch-flow tests against a real device and a real glove. Detox 20.51.3 is the React Native equivalent and stays unused unless we take the retreat path in §7.2.
App	sentry-dart	9.30.1	First-class SDK — but the free Developer plan is one seat and 5k events. A four-person team needs a paid plan or a shared seat, decided in Week 5, not discovered in Week 9.
Web — T3 desktop and T4 mobile browser (React/TypeScript)			

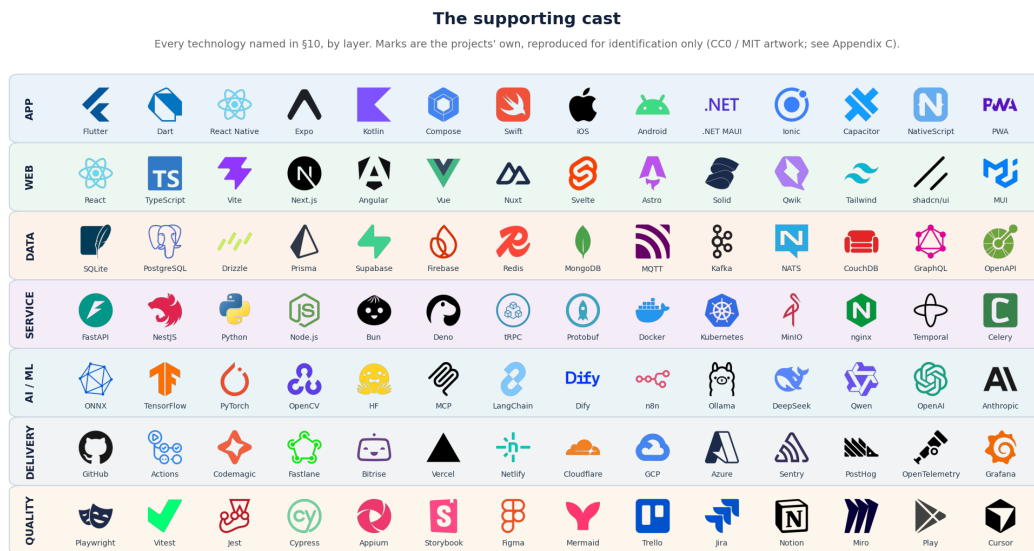
Layer	Component	Pin	What the pin means for LineSight
Web	React, React DOM	19.3.0 (9 Sep 2026)	React Compiler reached 1.0 in October 2025, so memoisation is generated rather than hand-written — it removes an entire class of Review-grid performance work. Governance moved to the React Foundation under the Linux Foundation in February 2026, which is the fact behind the longevity score in §8.1.
Web	TypeScript	7.0.2 (native Go compiler, GA Jul 2026)	An order-of-magnitude faster type-check. That is what makes whole-project checking on every pull request cheap, which is in turn what makes OpenAPI codegen survivable at our size.
Web	Vite	8.3.3 (Rolldown, GA Mar 2026)	One Rust bundler for both dev and build; the esbuild/Rollup split is gone. It is the majority choice for new projects in the 2025 State of JS survey, so this pin is a low-risk one.
Web	React Router	8.4.0	ESM-only. Remix's absorption into React Router 7 (2024) and the separate Remix 3 rewrite are why §9.6 treats meta-frameworks as a risk factor rather than a capability.
Web	TanStack Query / Zustand	5.104.1 / 5.0.15	Server-state and client-state kept separate; Zustand's monthly downloads now exceed Redux Toolkit's ecosystem-wide, but the argument for us is the smaller API, not the popularity contest.
Web	openapi-typescript / orval	7.13.0 / 8.40.0	Generated types and clients are committed and regenerated in CI, so a contract change appears in review as a diff in one file. This is the mechanism behind the single-source schema in §10.3.
Web	Tailwind CSS / shadcn ui	4.3.3 / CLI 4.21.3	Tokens are exported from the same Figma variables that drive the Flutter theme (§12.2). Two front ends stay one product because the colour values come from one place.
Web	Zod / Vitest / Playwright	4.6.5 / 5.0.3 / 1.63.0	Runtime validation at the API boundary alongside the compile-time types; the performance budget attaches to the Playwright job.
Web	onnxruntime-web	1.30.0 (14 Sep 2026)	WASM everywhere, WebGPU where available. T3 does not need it; the browser-side pre-filter sketched in §12.4 does, and its monthly download count has roughly multiplied eleven-fold in a year, which is the adoption signal behind choosing ONNX at all.
Centre — service, data and inference			
Service	Python	3.13	3.14 has been stable for a year, but the inference and database wheels we depend on lead the release curve rather than follow it. We move after the module, not during it.
Service	FastAPI	0.142.2	0.142.0 added native OpenTelemetry exporting, which removes a collector wrapper from the observability design in §10.9.
Service	PostgreSQL	18.6	Generated columns and logical replication give us the audit view now and the PowerSync upgrade path in §12.4 later without a schema rewrite.
Service	ONNX Runtime	1.30.0	The same graph the panel runs, one hop from the API (§10.4).
Delivery			
Delivery	GitHub Actions	hosted, unpinned	Runner minute pricing was revised on 1 January 2026; iOS builds therefore run on release tags only. That is a cost decision, recorded as ADR-008.
Delivery	Codemagic	Free tier, 500 build-minutes/month	Enough for a four-week sprint with caching. The plan is re-checked at Checkpoint 2, not at Week 12.
Delivery	Fastlane	2.240.1	Store metadata and screenshot automation, held in reserve behind Codemagic.
Delivery	TestFlight	100 internal / 10,000 external, 90-day expiry	The expiry is the trap: a build uploaded for the Week 7 checkpoint is dead before the Week 11 demo unless it is re-uploaded, so the re-upload is a scheduled Kanban card, not a memory.
Not adopted, and why — maintenance and licence signals			
Reject	Isar (Dart)	3.1.0+1, last release 2023	Fast and unmaintained. The C4 rejection in §7.1's alternatives is a maintenance fact, not a design opinion.
Reject	TensorFlow.js	4.22.0, last release Oct 2024	Superseded for our purposes by onnxruntime-web. Choosing a stale runtime is how a project discovers in Week 8 that its browser path has no WebGPU support.
Reject	EMQX open source	5.8; OSS support ended 28 Feb 2026	Later releases moved to the Business Source Licence. If the pilot needs a broker it is Mosquitto 2.1.2 (EPL) or NATS 2.15.0 (Apache-2.0), and §10.2's outbox means we do not need either yet.
Reject	Directus	12.4.1	Its licence is now the non-OSI Copyfair MSC terms. Any headless-CMS shortcut would have to be re-licensed before a real deployment, which is precisely the kind of debt a student project does not need.
Reject	Realm Device Sync	deprecated Sept 2024	The vendor's own signal, and the reason §10.1 kept it on the list of alternatives rather than the list of options.
Reject	RxDB	17.6	Free tier capped at 13 collections, with premium plugins paid. A sync engine whose pricing model we cannot predict is not the edge of a four-week plan.
Reject	YOLO26n	2.4M-parameter detector	Architecturally a good match for the panel, but AGPL-3.0 with a paid Enterprise licence for the weights. We export our own small detector to ONNX instead, so the shipping artefact carries no licence ambiguity — see the attribution rules in Appendix C.

Table 10 — Version baseline as at 7 October 2026, with the rejects recorded alongside the picks. Pins are the versions we branch from; nothing here is a claim that a later version does not exist on the day this is read.

[CAVEAT] Two blocks above are load-bearing for the argument rather than housekeeping. First, **the inference boundary is community-maintained on the app side**: the Dart binding bundles a runtime one minor behind the server's, so §10.4's "one graph, three targets" is exact about the *format* and approximate about the *runtimes*. That is a real weakness in a decision we have just defended, and it is measured by a spike rather than argued away. Second, **the browser cannot do what our offline story assumes**: Background Sync is Chromium-only at roughly 77% coverage, iOS web-app push arrived only at Safari 16.4 with a 4 KB payload cap and no silent delivery, and web storage is evicted under pressure. That is the technical content of §9.3's elimination, and the reason the T4 shell is designed read-mostly with the outbox on the server rather than as a second copy of the edge database.

10.9 Quality, observability and the evidence trail

Testing. Dart unit tests for the domain layer (the event model and the sync pump are pure logic and must be), Flutter integration_test plus **Patrol** or **Maestro** for the touch flows on a real device, **Vitest + Playwright** for the React side, and **pytest** against the service with a real Postgres container — never a mocked database, because the sync logic's whole purpose is to be right about transactions. **Observability.** **Sentry** on all three tiers (its Flutter and React SDKs are first-class) for crashes, **PostHog** for the Week 9 usability metrics the User Testing Report needs (10+ testers, funnel from install to first verdict), and structured JSON logs with the event ULID as the trace key so a complaint can be followed from a tap on a panel to a row in Postgres. **Evidence.** Every pipeline run, board movement and decision record is a graded artefact: the handbook's Weekly Process component is 40% of the mark and Checkpoint 2 asks for Kanban evidence, so the delivery pipeline is chosen partly for how legibly it produces that trail.



Shaded bands = the seven layers of the decision; the two chosen headline marks (Flutter, React) appear in their bands as candidates, not as endorsements.

Figure 3 — The supporting-cast mark sheet: every technology named in §10 with its logo, licence family and the layer it occupies. Full attribution and licence terms in Appendix C.

11 | Results

[EVIDENCE] Table 7 is the consensus app-side sheet. Rows are candidates, columns are the criteria from §5.1, and the final column is the weighted sum. The sheet is reproduced with all three independent scorer columns in Appendix A, so a mentor who disagrees with a cell can see who wrote it and re-run the total.

Candidate	C1	C2	C3	C4	C5	C6	C7	C8	C9	Total
	15%	12%	12%	10%	13%	13%	9%	8%	8%	/5.00
Flutter + Dart	4.0	5.0	4.5	4.5	3.0	4.0	4.0	4.0	4.5	4.14
React Native + Expo	3.5	3.0	4.5	3.0	5.0	4.5	4.5	5.0	4.0	4.09
Ionic + Capacitor	4.5	2.0	3.5	2.5	5.0	4.5	3.5	4.0	3.5	3.74
Native pair (Compose + SwiftUI)	2.0	5.0	5.0	4.5	2.0	2.0	4.5	3.5	5.0	3.56
PWA only	3.5	2.0	2.0	3.0	5.0	5.0	3.5	4.0	3.5	3.52
Kotlin/Compose Multiplatform	3.0	4.0	4.0	4.5	2.5	3.0	3.0	3.0	4.5	3.45
Electron / Tauri shell	1.5	4.0	3.5	3.0	4.5	3.5	3.5	3.5	3.5	3.34
NativeScript	3.5	2.5	3.0	2.5	3.5	3.0	2.0	2.0	2.0	2.78
Low / no-code platform	3.0	1.5	1.5	2.0	5.0	5.0	2.0	2.0	1.5	2.77
.NET MAUI	3.5	3.0	3.0	3.0	1.5	2.5	2.5	2.5	3.0	2.73

Table 7 — App-side weighted sheet. Highest total wins; the top two are separated by 0.055 points, which §11.3 shows is inside the noise of our own weighting choices, and §11.4 argues is the wrong place to look for the decision.

App-side weighted sheet: 10 candidates x 9 criteria										WEIGHTED TOTAL
Green = strength, red = weakness. Totals ranked at right; Flutter 4.14, React Native 4.09.										
Flutter + Dart	4	5	4.5	4.5	3	4	4	4	4.5	4.14
React Native + Expo	3.5	3	4.5	3	5	4.5	4.5	5	4	4.08
Ionic + Capacitor	4.5	2	3.5	2.5	5	4.5	3.5	4	3.5	3.73
Native pair	2	5	5	4.5	2	2	4.5	3.5	5	3.55
PWA only	3.5	2	2	3	5	5	3.5	4	3.5	3.52
KMP / Compose MP	3	4	4	4.5	2.5	3	3	3	4.5	3.44
Electron / Tauri	1.5	4	3.5	3	4.5	3.5	3.5	3.5	3.5	3.34
NativeScript	3.5	2.5	3	2.5	3.5	3	2	2	2	2.78
Low / no-code	3	1.5	1.5	2	5	5	2	2	1.5	2.77
.NET MAUI	3.5	3	3	3	1.5	2.5	2.5	2.5	3	2.73
	C1 Target coverage	C2 Rendering fidelity	C3 Device + camera	C4 Local-first data	C5 Team fit & ramp	C6 Time to W7	C7 Ecosystem depth	C8 Corpus / AI-ability	C9 Longevity & cost	

Figure 4 — The same sheet as a heat map. The structure is more informative than the totals: Flutter's column is flat and high, React Native's and Ionic's are spiky — strong exactly where our skills are, weak exactly where the Live View lives. The native pair and the PWA are mirror images of each other.

11.1 Ranked totals

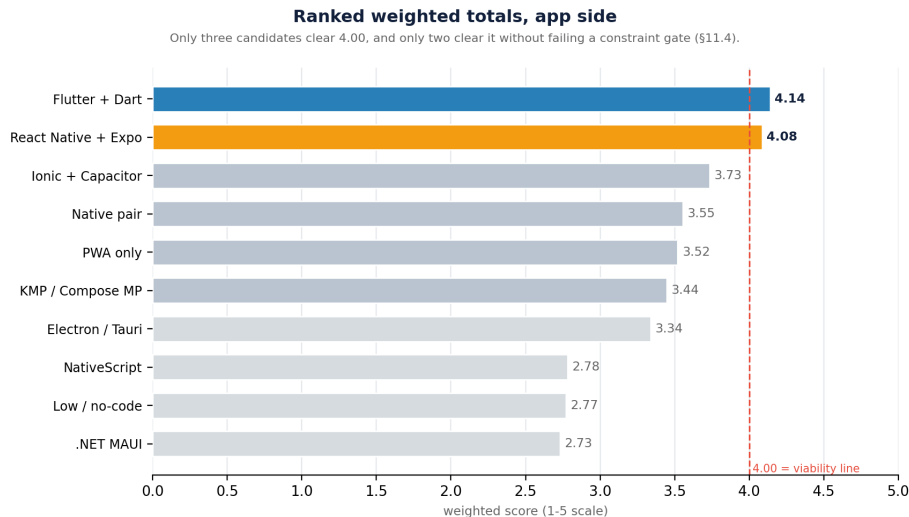


Figure 5 — Weighted totals with the top two called out. The four-point line is where a candidate becomes viable; exactly two clear it, and no candidate clears all three constraint gates of §11.4 unconditionally — the choice is which conditionality to buy out.

11.2 The web sheet

[METHOD] The web side is scored against six criteria with different weights, because the question is different: no rendering-critical surface, but real constraints on deployment inside a plant network and on the component ecosystem for data-dense views. Weights: W1 team skill reuse 25%, W2 data-dense component ecosystem 20%, W3 deployment simplicity 15%, W4 mobile-browser behaviour 10%, W5 public corpus and AI-assistability 15%, W6 longevity and governance 15%.

Candidate	W1	W2	W3	W4	W5	W6	Total
React + TypeScript + Vite	5.0	5.0	5.0	4.5	5.0	4.5	4.88
Next.js	4.5	5.0	3.0	4.5	5.0	4.5	4.45
Vue 3 + Nuxt	3.0	4.0	4.0	4.0	4.0	4.0	3.75
Remix / React Router 7	4.0	3.5	3.5	4.0	4.0	3.0	3.68
Angular	2.0	4.5	4.0	4.0	3.5	4.5	3.60
Svelte 5 + SvelteKit	2.0	3.0	4.5	4.0	3.0	3.5	3.15
Astro	3.0	2.0	4.5	3.5	2.5	3.5	3.08
SolidJS + SolidStart	2.0	2.5	4.0	4.0	2.0	3.0	2.75
Qwik	2.0	1.5	3.5	3.5	1.5	2.0	2.20

Table 8 — Web-side sheet. The React/Next pair separates on W3 alone: identical component access, different operational footprint.

11.3 Sensitivity: what would have to be true for a different answer

[METHOD] A weighted score is only as good as its robustness, so we stressed it three ways: re-weighting blocs, zeroing single criteria, and comparing the gap against our own scoring noise.

(i) The gap between first and second is smaller than our confidence in it.

Flutter leads React Native by 0.05 points on a 5-point scale where three people scored independently and disagreements of one point were common. On the raw sheets the two scorers who did not specialise in mobile put RN first; the app lead's C2 score is what produces the Flutter lead. We state this plainly rather than hiding it: **the weighted sheet does not decide this contest.**

(ii) Doubling the semester-realism bloc flips it.

Weighting C5 (team fit) and C6 (time-to-checkpoint) at twice their share — 26% each, everything else scaled down — gives React Native + Expo **4.22** against Flutter's **4.01**. A reader who believes a four-week deadline dominates rendering fidelity should choose React Native, and that is a coherent position. Our answer is (iv), not a re-argument of the weights.

(iii) Doubling the product-fidelity bloc widens it.

Weighting C2 and C3 at 24% each yields Flutter **4.26** against RN's **4.02**. Ionic never closes the gap under either scheme — 3.55 with the fidelity bloc doubled and 3.94 even with the realism bloc doubled, against a leader at 4.01–4.26 — because it is strong on the bloc we disagree about and weak on the criterion we do not.

(iv) Single-criterion zeroing (tornado).

Figure 6 shows the effect on the Flutter-minus-RN gap of removing each criterion's weight entirely. The gap is +0.055. Removing C2 (rendering fidelity) moves it to −0.185; removing C4 (local-first data) moves it to −0.095; removing C1 flips it by a smaller amount (−0.020), and removing C9 leaves a gap of only +0.015. Every other single removal leaves Flutter ahead. So the decision rests above all on two claims: **that Live View needs a painter, and that the event store needs a real database.**

Both are testable, both are scheduled in §13.2, and both have written pass gates — which is what a defensible stack decision looks like when the sheet is a coin flip.

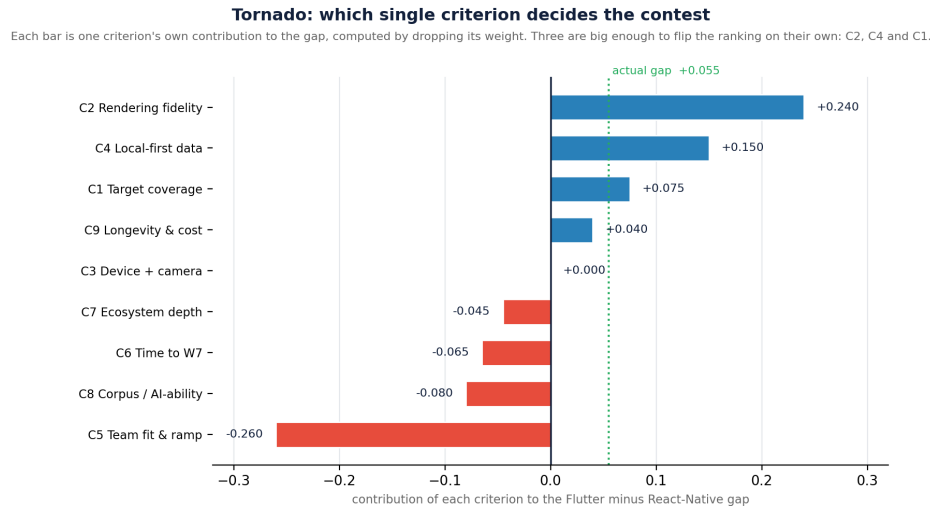


Figure 6 — Tornado analysis: change in the Flutter-minus-React-Native gap when each criterion's weight is set to zero. Three bars are long enough to flip the sign on their own (C2, C4 and C1); the first two are the claims the decision is built on.

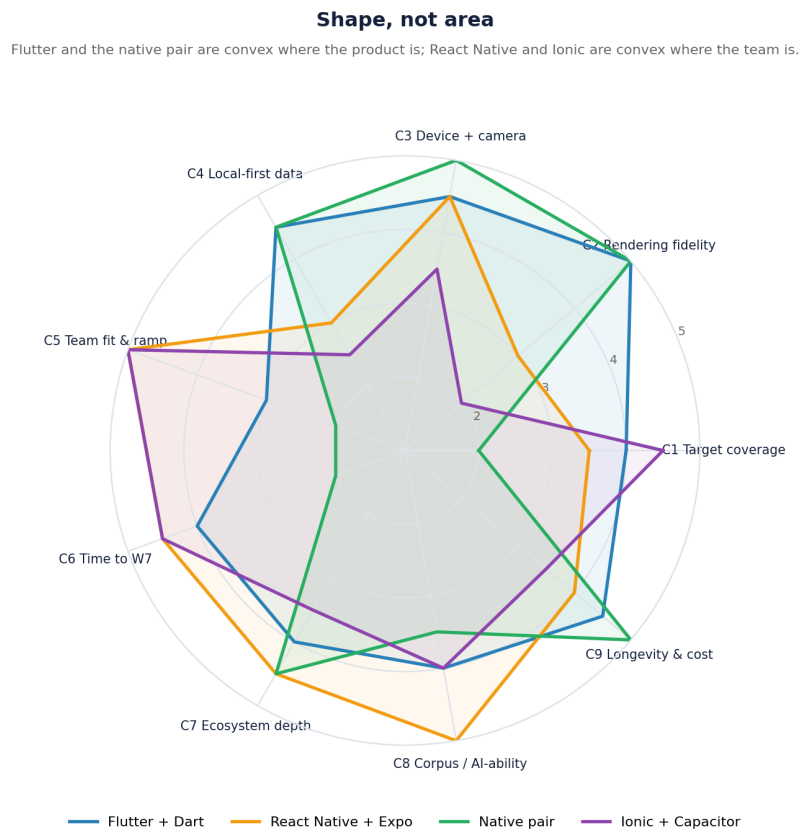


Figure 7 — The four surviving candidates on all nine criteria. Shape, not area, is the argument: Flutter and the native pair are convex where the product is, React Native and Ionic are convex where the team is.

11.4 Why the sheet is not the decision

[RATIONALE] Weighted averaging has a specific failure mode that matters here: it lets a fatal weakness be paid for with unrelated strengths. If a candidate cannot hold 50 fps while painting a camera frame with an overlay on the target panel, it does not matter that the team already knows its language — the product's central promise (a verdict inside a two-second loop, R3) is broken, and no amount of C5 score repairs it. So before the totals were read, two **constraint gates** were applied, and they are stated as numbers with dates rather than as opinions:

Gate	Requirement, stated as a measurement	Who clears it
G1 Painter	Live View sustains ≥ 50 fps p95 for 10 minutes with a 1280×720 camera feed, one bounding box, one banner and a scrolling event strip, on the reference panel	Flutter, native pair, KMP/Compose; conditional for RN (Fabric + canvas) and Ionic
G2 Store	A transactional, typed local database with a replayable outbox and per-station ordering, testable offline on a real device	Flutter (Drift), native pair, KMP (SQLDelight); conditional for RN and PWA
G3 Four weeks	Three features plus auth plus a demoable build by the Week 7 checkpoint, with the team's actual skills	RN, Ionic, PWA, low-code; conditional for Flutter and the native pair

Table 9 — Constraint gates. A candidate must clear G1 and G2 unconditionally, because they protect requirements we said we valued most; G3 is negotiable through scope, which is why it is the gate we accept risk on rather than the gates we do.

[DECISION] Flutter clears G1 and G2 by construction and is conditional on G3; React Native is the mirror image. **We buy G3 with scope and pairing rather than trading G1**, for one reason: G1's failure is discovered late and is unrecoverable inside the semester, while G3's failure is discovered in Week 5 and is recoverable by cutting a screen. That asymmetry of regret — not the 0.05-point sheet — is the actual argument, and \$12.3 attaches a kill criterion to it so that we cannot quietly believe it forever.

12 | The decision and what it commits us to

12.1 Statement

[DECISION] LineSight ships two front ends and one contract. The operator-facing application (T1 Android panel/handset, T2 iOS) is built in **Flutter** with **Dart**. The engineering- and supervisor-facing web application (T3 desktop browser, T4 mobile browser) is built in **React** with **TypeScript**, bundled by **Vite** as a progressive web app. Both consume one OpenAPI document generated by one **FastAPI** service over **PostgreSQL**; the app additionally owns a **SQLite** store with a replayable outbox. Inference is a single ONNX graph executed on the panel or in the service. Agents are server-side, speak MCP to the same API, and write nothing without a human confirmation.

[SCOPE] Just as important is the negative space. We are **not** using Flutter for the web surfaces; not writing a native Android or iOS module in Week 4–7 unless a gate forces it; not adopting a device-sync service, a GraphQL gateway, or an agent orchestration framework before Checkpoint 2; and not supporting iOS below the current App Store floor or Android below API 26 on the panel.

12.2 The architecture this produces

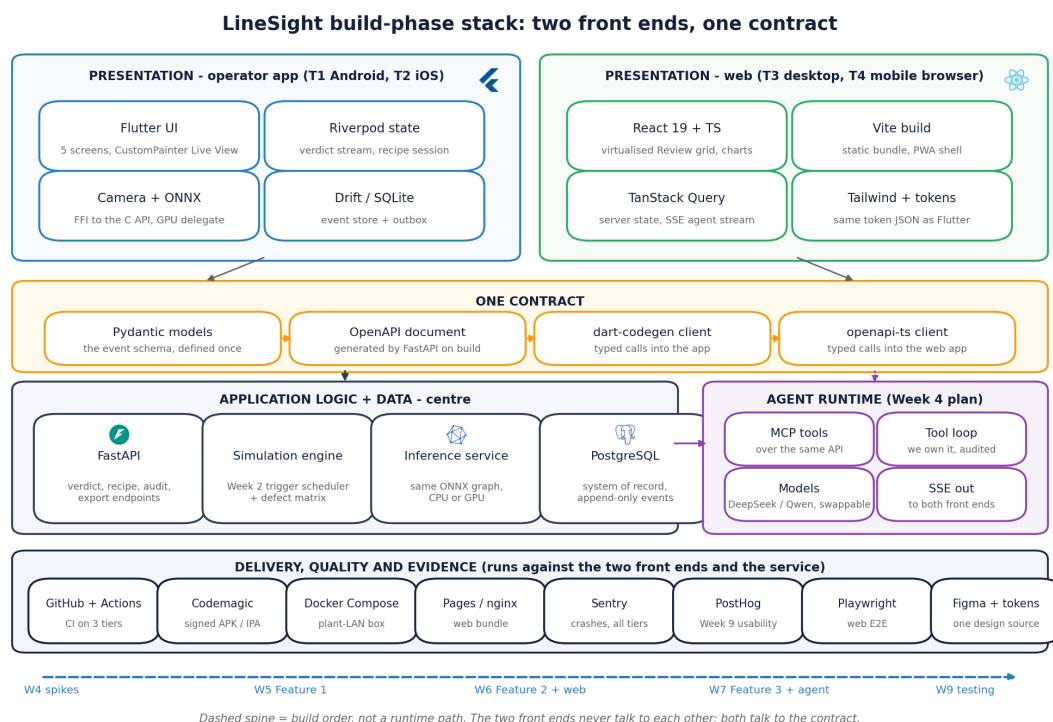


Figure 8 — The chosen stack, drawn against the Week 2 three-layer architecture it must implement. Presentation is two codebases; application logic and data are one contract with two projections. The dashed arrow is the Week 4–7 build order, not a runtime path.

[ARCHITECTURE] Three properties of this shape are worth defending to a mentor, because each is a place where a different team might reasonably disagree with us.

(i) Two front ends, one source of truth about the domain. The event schema exists once, in Python, with Pydantic models as the definition and an OpenAPI document as the export. Code generation produces TypeScript types and client calls for React and Dart types and clients for Flutter. A schema change cannot silently reach one front end: the other fails to compile, which

is the cheapest possible failure. This is our structural answer to the three-codebase problem §2 identifies, and it is the reason the web team's and app team's definitions of "event" cannot drift apart between Week 5 and Week 9.

(ii) Business rules live below the UI, in a form both front ends can call. Verdict thresholds, recipe compatibility checks, containment rules and audit semantics are service-side functions with a pure, tested core; the Flutter app mirrors the ones it needs offline, generated from the same specification rather than hand-copied. If we later adopt Kotlin Multiplatform for a shared logic layer — the migration §7.3 keeps open — the seam is already there.

(iii) Design tokens, not design duplication. The Figma library from Weeks 2–3 exports a token file (colours, spacing scale, type ramp, touch-target minimums, contrast pairs). A build step turns it into a Dart ThemeData and into CSS custom properties consumed by Tailwind. When the Week 9 usability round says "the FAIL state is not visible under glare," one token changes and both surfaces move — which is the only realistic way two codebases stay one product.

12.3 Kill criteria — the conditions under which we abandon this decision

[RISK] A decision without a kill criterion is a belief. Two are attached, both with dates, both with a named owner, and both written before the measurements exist:

#	Trigger	Deadline	Action
K1	Live View on the reference panel holds <50 fps p95 with camera feed + overlay + event strip, after two days of optimisation (texture pooling, reduced paint area, separate raster layer)	18 Oct 2026 (end W5)	Move Live View to an Android-native surface behind a platform channel, or invoke the RN retreat (below)
K2	The team cannot land Feature 1 (auth + verdict + event write) without the app lead personally reviewing every Dart file, i.e. Dart has not become a team skill	25 Oct 2026 (end W6)	Invoke the RN retreat: React Native + Expo, reusing the web components and the generated TypeScript client
K3	iOS signing/pipeline cannot produce a TestFlight build	25 Oct 2026	Ship iOS as a documented limitation; demo on Android and web; do not spend Week 7 on certificates

[NOTE] The RN retreat is bounded and pre-planned, which is what makes taking Flutter now affordable: roughly 60% of the app's non-UI logic (the generated client, the outbox semantics, the domain rules) survives a move to React Native, and the screens would be rewritten once. Cost estimated at eight to ten team-days — expensive, survivable, and far cheaper than discovering in Week 9 that the verdict surface stutters.

12.4 What we gave up, and the upgrade path we kept

Given up	What it would have bought	Revisit when
React Native's developer experience	Near-zero ramp, Expo's managed builds and OTA updates, and the largest AI-legible corpus of any mobile option	K2 fires, or if we add a second app surface that is form-shaped rather than painter-shaped
Ionic's single codebase	One artefact for all four targets and total skill reuse	Never for the HMI; plausible for a public-facing informational page
Flutter web	One language across all four targets	Never for the data-dense surfaces; plausible for a station status board rendered as a canvas
PowerSync / Electric-SQL device replication	Continuous Postgres-to-device sync with conflict handling we do not maintain. Of the two, PowerSync ships a maintained Flutter SDK (v2.4 line, Apache-2.0) while Electric does not, which pre-decides the choice if this row ever fires	Station count exceeds two, or the pilot runs across shifts with real MES integration
GraphQL gateway	One typed query layer for both front ends and field-level selection for the Review grid	Week 8+, only if the REST surface shows N+1 round-trip problems in the field
Agent orchestration framework (Lang-Graph)	Checkpointed multi-step agent runs, replay, human-in-the-loop primitives	When an agent surface needs more than one tool call per turn

12.5 Deployment topology

[STACK] Three environments, deliberately boring. **(1) Laptop/dev:** Docker Compose with Postgres, the FastAPI service, a MinIO bucket for frame fixtures, and the Vite dev server proxying to it. **(2) Plant pilot:** the same Compose file on a box the plant lends us, nginx serving the built web bundle on the LAN, panels running the released APK in kiosk mode and talking to the LAN address; the whole thing keeps working with the internet off, which is the property R7 exists to protect. **(3) Demo/promotion:** the web bundle on Cloudflare Pages or Netlify, the service on a free-tier container host or a university VM, a seeded dataset so a stranger's first screen has content. Assessors get a URL, a QR code to the APK on a hosted page, and a seeded demo login — satisfying R11 without anyone needing a developer account.

12.6 Risk register for the stack (product risks live in the Week 2 register)

ID	Risk	L	I	Mitigation / owner
S1	Dart ramp consumes Week 5; Feature 1 slips	M	H	Pair-programmed spikes; generated clients remove hand-written plumbing; K2 date; owner: app lead
S2	Live View misses the frame budget on the actual panel	M	H	K1 gate; fallback to platform-native camera surface; reduce overlay complexity before reducing correctness
S3	iOS certificates and provisioning eat a week	H	M	Start signing in Week 4 as a task with its own Kanban card; Codemagic or a borrowed Mac; K3 allows a documented iOS limitation
S4	Two front ends drift in behaviour and the demo shows two products	M	M	Token pipeline, shared OpenAPI, a cross-surface acceptance checklist run before each checkpoint
S5	A plugin we depend on is unmaintained	M	M	Plugin due-diligence rule: last release <12 months, one working example we run on the panel, and a named fallback; no exotic plugins before Checkpoint 2
S6	Agent answers are confidently wrong in front of a user	M	H	Tools return only data the API already exposes; every answer carries a provenance link; no agent write path without confirmation; disclosure in the UI
S7	Free-tier limits or a regional API block stop the demo	M	M	Two model providers behind one interface; a scripted fixture replay mode that needs no network at all
S8	Scope exceeds four part-time people	H	H	Feature order fixed in §13.3; anything not on the Week 7 list is Week 8+; the mentor reviews the list, not the team

Table 11 — Stack risk register. $L \in \{L, M, H\}$ likelihood, $I \in \{L, M, H\}$ impact. Figure 9 plots the same data; S8 is the risk that a semester project actually dies of.



Figure 9 — Stack risk heat map.

13 | From decision to running code

13.1 Team shape and the pairing rule

[DELIVERABLE] Four people, two front ends, one service. **A (app lead)** owns Flutter, the panel, the camera path and gate G1. **B (web/data lead)** owns React, the token pipeline, the generated clients and the Review grid. **C (service/agent)** owns FastAPI, Postgres, the outbox pump, the ONNX export and the agent runtime. **D (product/quality)** owns personas, the test script, the evidence trail, and the Week 8–9 promotion and testing artefacts. The pairing rule exists because of risk S1: every Flutter pull request is reviewed by a non-Flutter author, and D writes at least one screen per week in Dart. If that rule stops working, K2 has already fired and we simply have not noticed yet.

13.2 Week 4 spikes, with pass gates

[METHOD] Five time-boxed spikes run this week, before any feature work, each with a numeric gate and a named owner. They are cheap (a total of roughly six team-days) and they are what converts §11.4's two claims from assertions into measurements.

#	Spike	Pass gate	Owner
SP1	Flutter Live View on the reference Android panel: camera preview, one tracking box, verdict banner, scrolling event strip	≥ 50 fps p95 over 10 min; ≤ 180 ms glass-to-glass	A
SP2	Offline outbox: 500 events written with the network off, replayed on reconnect, then replayed again after a forced crash	Zero duplicates, correct permutation order, crash recovery in < 5 s	C

#	Spike	Pass gate	Owner
SP3	ONNX inference on the panel via Dart FFI, small detector at the station's resolution	≤400 ms per frame on CPU, ≤150 ms with GPU delegate; else move inference to the service	A/C
SP4	Token pipeline: Figma library → JSON → Dart ThemeData and CSS custom properties, verified on one screen in both front ends	Same computed colours and spacing on both, checked with a screenshot diff	B
SP5	Agent vertical slice: one MCP tool over the events API, streamed to a Flutter widget with SSE	A correct, cited answer about seeded data in <6 s; graceful failure with no network	C

13.3 Build order to Checkpoint 2 and Demo Day

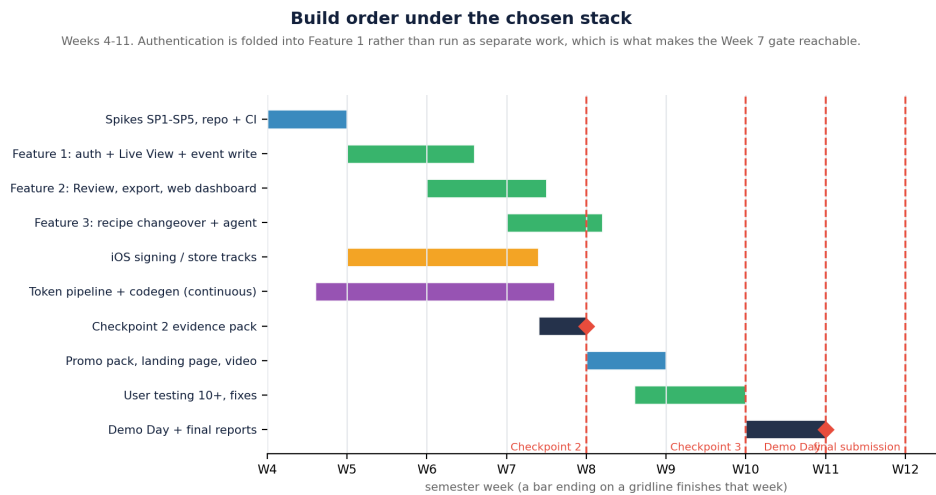


Figure 10 — Weeks 4–10. The three features due at Checkpoint 2 are the first three bars of Week 5 onward; the handbook's Week 5 authentication requirement is folded into Feature 1 rather than treated as separate work.

Week	Exit criterion (what a mentor can see)	Handbook deliverable
4	This document; repo with CI green on three tiers; five spikes closed with measurements	Tech stack decision; agent integration plan; Draft 1
5	Feature 1: badge login + Live View verdict on the panel + one event in the store, offline	Working login + Feature 1
6	Feature 2: Review & History with filters, replay of a stored frame, CSV export; web dashboard live on the LAN	Working app with 2 features
7	Feature 3: recipe changeover with the guided checklist and version guard; first agent surface (defect-cluster explanation)	Checkpoint 2: app, Kanban evidence, stack rationale, reflection
8	Promo pack: landing page, 1-minute video, shareable demo environment with seeded data	Promotion report inputs
9	User testing with 10+ participants across both front ends; fixes shipped; analytics evidence	Draft 2; Checkpoint 3
10	Demo Day: live scenario on a real panel, projected web dashboard, recorded video	Final reports

[NEXT] Deliberately excluded before Checkpoint 2, so that the exclusion is on the record rather than a surprise: MES integration beyond a CSV contract, multi-station replication, an offline map of the plant, model retraining in-app, iOS-specific polish beyond functional parity, and any second agent surface. Each has a home in Week 8–10 or in the limitations section of the final report, which is a better place for an honest list than a demo that quietly omits it.

14 | Architecture decision records

[METHOD] Nine records, written in the standard form (context, decision, options considered, consequences) and dated. They exist because the module's own evidence requirements — “tech stack rationale” at Checkpoint 2, “known limitations” in the final report — are answered best by a log rather than by a narrative written after the fact, and because a decision record that names its rejected options is the only artefact that stops a team re-litigating the same choice in Week 6.

ID	Title / status	Decision, rejected options, accepted cost
ADR-001	App framework for T1/T2 Accepted 07 Oct 2026	Flutter 3.4x stable, Dart. Rejected: React Native + Expo (sheet 4.09 vs 4.14; fails G1 confidence on the reference panel), native pair (three codebases, no iOS skill), KMP/Compose (shared-logic story we may adopt later), Ionic (renderer), MAUI (ramp + trajectory), low-code (no painter). Cost accepted: a new language for four people, paid for with frame-time predictability and a typed local store.
ADR-002	Web framework for T3/T4 Accepted 07 Oct 2026	React 19 + TypeScript, bundled by Vite, as a PWA-style SPA. Rejected: Next.js (server runtime buys us nothing behind a login), Angular and Vue (skill write-off), Svelte/Astro/Solid/Qwik (component ecosystem for dense grids). Cost accepted: no SSR/SEO for the product; a separate static landing page in Week 8 if promotion needs it.
ADR-003	Edge data store Accepted 07 Oct 2026	SQLite via Drift (typed DAO, compile-time query checks, reactive streams), schema generated from the service's OpenAPI/Pydantic definition. Rejected: Isar (maintenance), Realm (vendor conflict model, narrowing sync line), Hive (no queries), platform key-value (cannot express R6–R8). Cost accepted: we own migrations and the pump.
ADR-004	Offline sync approach Accepted 07 Oct 2026	Outbox with client-generated ULIDs, per-station sequence, server-authoritative time, no merge. Rejected: PowerSync/ElectricSQL (right answer for eight stations, wrong week), CRDTs (Automerge/Yjs — no collaborative-editing requirement), last-write-wins without a history (breaks the audit promise). Cost accepted: conflict policy is a product decision we must write down rather than inherit.
ADR-005	Central service Accepted 07 Oct 2026	Python + FastAPI + PostgreSQL in Docker Compose. Rejected: NestJS (3.4 vs 4.0 on the layer sheet; would mean rewriting the Week 2 engine), Supabase/Firebase as primary (a hosted backend as the system of record conflicts with the plant-LAN deployment and the audit story), serverless functions (background work and long inference calls). Cost accepted: we run a database.
ADR-006	Inference boundary Accepted 07 Oct 2026	One ONNX graph, executed on-device through Dart FFI or in the service; onnxruntime-web where the browser needs a preview. Rejected: LiteRT-only (excellent on Android, weaker cross-target), Core ML (iOS-only second path), server-only inference (fails R7 when the network is the thing that is down). Cost accepted: export discipline and a quantisation step we must not skip.
ADR-007	Agent substrate Accepted 07 Oct 2026	Server-side tool loop we own, MCP for tool exposure, two model providers behind one interface, SSE transport, human confirmation on every write. Rejected: client-side agents (no auditability), a heavyweight orchestration framework before Checkpoint 2, prompt-only designs with no tools (they hallucinate numbers). Cost accepted: we maintain the loop and the tool descriptions.
ADR-008	Delivery pipeline Accepted 07 Oct 2026	GitHub + Actions for CI on all tiers; Codemagic for signed mobile builds; internal testing track and sideloaded APK for the panel; static hosting for web; Docker Compose for the service. Rejected: hand-run fastlane (learning cost), a self-hosted runner farm (no budget), store-wide public release (not needed for a pilot). Cost accepted: macOS minute billing on release candidates.
ADR-009	Repository layout Accepted 07 Oct 2026	Monorepo: /apps/flutter, /apps/web, /services/api, /packages/contract (OpenAPI, tokens, generated clients), /docs/adr, /evidence. Rejected: four repositories (the contract change must be atomic across surfaces; four repos turn that into a coordination project). Cost accepted: CI needs path filters to stay fast.

14.1 Evidence trail and how this document is graded

Assessment element	What this week produces	Where
Weekly Process (40%)	Kanban cards for the nine ADRs and five spikes, with movement history; three signed scoring sheets; a decision log entry per ADR; CI runs on three tiers	§13, Appendix A
Final Deliverables (30%)	Draft 1 of the Digital Prototype Report consumes §1–§5 and §12; Checkpoint 2 (Week 7) consumes §11, §13.3 and the spike measurements as its “tech stack rationale”	This document
Demo Day (20%)	The stack must produce a live scenario on a real panel plus a projected web dashboard; §12.5 exists so the demo is a deployment, not an improvisation	§12.5, §13.3
Journey Reflection (10%)	The kill criteria in §12.3 and the honest 0.05-point gap in §11.3 are the material for a reflection that says what we got wrong, not what we got right	§11.3, §12.3

Appendix A | Raw scoring sheets

Three independent sheets, then the consensus used in Table 7. Cells where the spread exceeded one point are marked with a discussion flag and were re-scored once, together, with the evidence for the cell written into the sheet.

Candidate	C1 target coverage			Con.	C2 rendering fidelity			Con.
	A	B	C		A	B	C	
Flutter	4	4	4	4.0	5	5	4	5.0
React Native + Expo	4	3	3	3.5	3	3	4	3.0
Ionic + Capacitor	5	4	4	4.5	2	2	2	2.0
Native pair	2	2	2	2.0	5	5	5	5.0
PWA only	4	3	3	3.5	2	2	2	2.0
KMP / Compose MP	3	3	3	3.0	4	4	4	4.0
Electron / Tauri	2	1	1	1.5	4	4	4	4.0
NativeScript	4	3	3	3.5	3	2	2	2.5
Low / no-code	3	3	3	3.0	2	1	1	1.5
.NET MAUI	4	3	3	3.5	3	3	3	3.0

Table A1 — Extract: two criteria across three scorers (A app lead, B web/data, C product/quality). The full nine-criterion grid for all three sheets is maintained in the repository as evidence/scoring.csv so that totals can be recomputed with any weights.

Candidate	C3	C4	C5	C6	C7	C8	C9
Flutter	4.5	4.5	3.0	4.0	4.0	4.0	4.5
React Native + Expo	4.5	3.0	5.0	4.5	4.5	5.0	4.0
Ionic + Capacitor	3.5	2.5	5.0	4.5	3.5	4.0	3.5
Native pair	5.0	4.5	2.0	2.0	4.5	3.5	5.0
PWA only	2.0	3.0	5.0	5.0	3.5	4.0	3.5
KMP / Compose MP	4.0	4.5	2.5	3.0	3.0	3.0	4.5
Electron / Tauri	3.5	3.0	4.5	3.5	3.5	3.5	3.5
NativeScript	3.0	2.5	3.5	3.0	2.0	2.0	2.0
Low / no-code	1.5	2.0	5.0	5.0	2.0	2.0	1.5
.NET MAUI	3.0	3.0	1.5	2.5	2.5	2.5	3.0

Table A2 — Consensus scores for the remaining seven criteria.

Appendix B | Glossary for a non-mobile reader

Term	Plain meaning as used here
Ahead-of-time (AOT) compilation	Source is compiled to machine code before it runs, so startup and frame times are predictable, at the cost of a larger binary.
Bridge (RN)	The message-passing layer between JavaScript and native views in React Native's original design; the reason per-frame custom drawing is expensive there. JSI/Fabric replace most of it.
Canvas / self-painting UI	A framework that draws every pixel itself rather than composing the operating system's widgets — Flutter's and Compose's model.
Hot reload	Injecting changed code into a running app and redrawing without losing screen state; the main reason mobile development feels fast.
Hydration / resumability	Re-running server-rendered markup on the client (hydration) versus serialising the client's state from the server so it does not have to (resumability, Qwik).
Idempotency key	A client-generated unique ID that makes a retried write safe: the server sees the same key and does not create a second record.
Jank / frame budget	A dropped frame the eye sees as stutter. At 60 Hz the budget is 16.7 ms per frame for everything: decode, layout, paint, composite.
Kiosk / device-owner mode	Android's facility to lock a device to one app with no status bar — what a line-side panel needs, and what a browser cannot do.
Outbox pattern	Writing the business record and its "needs sending" marker in one local transaction, then draining it in the background; the standard offline-first answer to "did the server get it?".
PWA	Progressive web app: a website with a manifest and service worker so it can be installed, launches offline, and receives push where the platform allows it.
SSR / SSG / RSC	Rendering on the server per request, at build time, and as a server-side component tree that ships serialised output to the browser (React Server Components).
WebView	The embedded browser inside a native app that Ionic and Capacitor render into; convenient, and the reason their UI inherits browser performance characteristics.

Appendix C | Logo marks, sources and licence terms

[NOTE] Every mark reproduced in this document is the property of its project or company and is shown solely to identify the technology named — an editorial, non-endorsing use. The vector artwork we render the marks from is itself freely licensed, as set out below; brand colours are taken from the same upstream metadata files. No mark is presented as LineSight's, and no mark implies that its owner recommends this project.

Group	Marks shown	Artwork source	Licence terms as relied on
App frame-works	Flutter, Dart, React/React Native, Kotlin, Jetpack Compose, Swift, .NET, Ionic, Capacitor, NativeScript, Expo	Simple Icons 16.x (single-path SVG), Devicons for the multicolour variants	Simple Icons: CC0-1.0 for the path data; the underlying brand mark remains a trademark of its owner. Devicons: MIT, with brand marks similarly owned by their projects.
Web frame-works & tooling	Vite, Next.js, Angular, Vue, Nuxt, Svelte, Astro, SolidJS, Qwik, Remix, React Router, Tailwind CSS, shadcn/ui, MUI, Ant Design, Chakra, Mantine, Radix, Bootstrap, TypeScript, JavaScript, Node.js, Bun, Deno, esbuild, Rolldown, webpack	As above	As above; Tailwind, shadcn/ui and Radix additionally publish their own official brand assets under permissive terms.
Data & services	PostgreSQL, SQLite, Supabase, Firebase, MongoDB, Redis, CouchDB, MQTT, Mosquitto, Kafka, NATS, Hasura, Directus, Prisma, Drizzle, GraphQL, OpenAPI, Protocol Buffers, tRPC, TanStack, Redux, React Query, SWR	As above	As above. PostgreSQL's elephant and SQLite's feather are project-owned marks reproduced here for identification only.

Group	Marks shown	Artwork source	Licence terms as relied on
Inference & AI	ONNX, TensorFlow, PyTorch, OpenCV, Hugging Face, OpenAI, Anthropic, Claude, DeepSeek, Qwen, Gemini, Ollama, MCP, LangChain, Dify, n8n	As above; OpenAI and Playwright marks taken from the archived Simple Icons v11 dataset, since those marks were withdrawn from current releases at the owner's request	As above, with the added note that a withdrawn mark is reproduced here only in this identification role and should be replaced or removed before any public release of this document.
Delivery & operations	Docker, Kubernetes, GitHub, GitHub Actions, GitLab, Fastlane, Codemagic, Bitrise, Vercel, Netlify, Cloudflare, Google Cloud, Azure, Sentry, PostHog, Grafana, Prometheus, OpenTelemetry	As above	As above.
Testing & design	Playwright, Cypress, Jest, Vitest, Appium, Figma, Mermaid, Trello, Jira, Notion, Miro, PWA, Android, Apple, Google Play, Electron, Tauri	As above	As above.
Institution marks	XJTLU logo in the running header and footer	Supplied by the university's programme materials	Used under the module's own guidance for student coursework; not a brand asset of this project.

[NEXT] If this document is published outside the cohort — on the Week 8 landing page, in a promotion post, or in a public repository — the marks in the “Inference & AI” row should be replaced with each project’s own brand-asset download from its official site, or dropped in favour of plain text names. We note this now so it is not discovered by a marketing team later.

15 | References

[HOW TO READ THIS] Sources are grouped by the section that uses them, so a claim can be checked without hunting. **Internal inputs** are the Week 1–3 LineSight deliverables this decision is required to build on. **Primary project sources** are the official documentation, specifications and release notes of the tools named; every version, date and price stated in this document was read from them on **7 October 2026**, via the public registries (pub.dev, npm, PyPI, GitHub releases) and the vendors’ own release notes. **Surveys and market data** are the third-party evidence behind the corpus and adoption arguments in §5 and §11. No design claim here rests on a secondary blog post: where one mattered, the framework’s own documentation or the language specification was read instead.

15.1 Internal inputs to this decision

- **[I1]** ENT207TC Module Handbook 2026–27 (final v3) — Weeks 4–7 deliverable lists, the Checkpoint 2 submission requirements, the Weekly Process component (40%), and the “platform of choice” wording that makes a low-code route legitimate. Local copy: ENT207TC_ModuleHandbook_2627_Final3.pdf.
- **[I2]** ENT207TC Week 4 briefing, “From prototype to build” — the Checkpoint Draft Workshop brief for Choose Stack, and the AI-agents workshop framing that requires an agent integration plan. Local copy: ENT207TC Week 4 From prototype to build.pdf.
- **[I3]** LineSight Week 2 deliverable — personas and the requirement register (R1–R15), the three-layer architecture, the mmd_exceptions and event-schema design including the sync-status column and the audit view, the simulation engine, and the competitive analysis that identifies traceability as the enterprise purchase driver. Local copy: week2/LineSight_Week2_Deliverable.pdf.
- **[I4]** LineSight Week 3 outputs — the Figma component library and design tokens that §12.2 turns into a two-target token pipeline, and the Checkpoint 1 demo recording whose screens are re-used as the fixtures in §13.

15.2 App-side candidates (§6, §7, §9)

- **[A1]** Flutter and Dart — release notes and cadence <https://docs.flutter.dev/release/release-notes>; the Dart specification and ECMA-408 <https://dart.dev/language>, <https://ecma-international.org/publications-and-standards/standards/ecma-408/>; the Impeller engine design notes <https://docs.flutter.dev/perf/impeller>; the CustomPainter API behind Live View <https://api.flutter.dev>; origin material (the 2011 Dart announcement, the 2015 Flutter/Sky reveal) in <https://dart.dev/blog> and <https://blog.flutter.dev>.
- **[A2]** React Native and Expo — the New Architecture (Fabric, JSI, Turbo Modules, the 0.76 default switch) <https://reactnative.dev/architecture>; Hermes <https://reactnative.dev/docs/hermes>; Expo’s managed workflow, EAS Build, dev clients and over-the-air updates <https://docs.expo.dev>; VisionCamera (the C3 score in §7.2) <https://github.com/mrousavy/react-native-vision-camera>; Reanimated, the off-JS-thread runtime behind C2 <https://docs.swmansion.com/react-native-reanimated/>; and <https://react.dev> for the hooks and compiler model both assume.
- **[A3]** Kotlin Multiplatform and Compose Multiplatform — <https://kotlinlang.org/docs/multiplatform.html>, <https://www.jetbrains.com/compose-multiplatform/>, SQLDelight <https://sqldelight.github.io/s>



qldelight/, Ktor <https://ktor.io>; the 2019 Kotlin endorsement and Compose 1.0 at <https://developer.android.com/jetpack/compose>.

- **[A4]** The native pair — Jetpack Compose <https://developer.android.com/develop/ui/compose>, Camera2 performance guidance <https://developer.android.com/media/camera>, Swift <https://www.swift.org>, SwiftUI <https://developer.apple.com/xcode/swiftui/>, Core ML <https://developer.apple.com/machine-learning/core-ml/>, and device-owner/kiosk behaviour <https://developer.android.com/work/dmg>.
- **[A5]** Eliminated app-side options, cited because §9 records them — .NET MAUI <https://learn.microsoft.com/dotnet/maui/>; Ionic and Capacitor <https://ionicframework.com/docs>, <https://capacitorjs.com>; NativeScript <https://nativescript.org>; Electron <https://www.electronjs.org>; Tauri <https://tauri.app>; the PWA specifications (service workers, manifests, Background Sync, Push) https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API, https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API, https://developer.mozilla.org/en-US/docs/Web/API/Push_API; and Apple's own account of what web apps do and do not get on iOS <https://webkit.org/blog/11274/> with the Safari release-notes archive <https://developer.apple.com/documentation/safari-release-notes>.

15.3 Web-side candidates (§8, §9.6)

- **[W1]** React <https://react.dev> (19.x release notes, the React Compiler documentation, the transition to Foundation governance); TypeScript, including the native Go-language-service port <https://www.typescriptlang.org>, <https://devblogs.microsoft.com/typescript/>; Vite and Rolldown <https://vite.dev>, <https://rolldown.rs>.
- **[W2]** The alternatives, each cited for the property §8–§9 relies on — Next.js <https://nextjs.org/docs>; React Router and the Remix merger <https://reactrouter.com>; Vue <https://vuejs.org>, Nuxt <https://nuxt.com>; Angular's zoneless/signals direction <https://angular.dev>; Svelte 5 runes <https://svelte.dev/docs> and SvelteKit <https://svelte.dev/docs/kit>; Astro's islands <https://docs.astro.build>; Solid <https://www.solidjs.com/docs>; Qwik's resumability <https://qwik.dev>.
- **[W3]** The supporting web layer named in Table 10 — Tailwind CSS <https://tailwindcss.com>; shadcn/ui <https://ui.shadcn.com>; TanStack Query and Table <https://tanstack.com>; Zustand <https://github.com/pmndrs/zustand>; Zod <https://zod.dev>; openapi-typescript <https://openapi-ts.tools>, orval <https://orval.dev>; Ant Design <https://ant.design> and MUI <https://mui.com> as the two grid ecosystems scored on W2.

15.4 Data, service, inference and agents (§10)

- **[D1]** SQLite <https://sqlite.org>, and the WAL and busy-timeout behaviour that §10.1's transactional outbox depends on <https://sqlite.org/wal.html>; Drift <https://drift.simonbinder.eu>; sqlite and the Dart SQLite bindings <https://pub.dev/packages/sqlite>.
- **[D2]** Replication engines, treated as the upgrade path in §12.4 — PowerSync <https://docs.powersync.com>, Electric <https://electric-sql.com>, WatermelonDB <https://nozbe.github.io/WatermelonDB/>, RxDB <https://rxdb.info> (its licence and collection caps are why §10.1 declines it), Automerge <https://automerge.org> and Yjs <https://yjs.dev>, named in §10.2 as the dependency we chose not to take.
- **[D3]** Central service — FastAPI <https://fastapi.tiangolo.com>; Pydantic <https://docs.pydantic.dev>; PostgreSQL <https://www.postgresql.org/docs/>; the OpenAPI specification and its Linux Foundation governance <https://spec.openapis.org>, <https://www.openapis.org>; Docker Compose <https://docs.docker.com/compose/>; nginx <https://nginx.org/en/docs/>.
- **[D4]** Inference, the substance of §10.4 — ONNX <https://onnx.ai>; ONNX Runtime, its execution providers and the WebGPU/WebAssembly compute paths <https://onnxruntime.ai/docs>; the Dart FFI that reaches that C API <https://dart.dev/interop>; LiteRT <https://ai.google.dev/edge/litert>, MediaPipe <https://developers.google.com/mediapipe>; Apple Core ML <https://developer.apple.com/machine-learning/>; WebGPU and WebCodecs on MDN https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API, https://developer.mozilla.org/en-US/docs/Web/API/WebCodecs_API, with coverage from <https://caniuse.com> — the source of the “not yet Baseline” claim about WebGPU; and WebNN's status as a Candidate Recommendation Draft implemented by no shipping browser <https://www.w3.org/TR/webnn/>.
- **[D5]** Agents — the Model Context Protocol specification and revision history <https://modelcontextprotocol.io>; LangGraph <https://langchain-ai.github.io/langgraph/>; the OpenAI Agents SDK <https://openai.github.io/openai-agents-python/>; DeepSeek and Qwen API and pricing <https://api-docs.deepseek.com>, <https://github.com/QwenLM>; SSE <https://html.spec.whatwg.org/multipage/server-sent-events.html>.

15.5 Delivery, quality and observability (§10.9, §13)

- **[P1]** GitHub Actions, including the runner and minute-multiplier pricing behind the iOS cost note in §10.8 <https://docs.github.com/en/actions>, <https://github.com/pricing>; Codemagic <https://docs.codemagic.io>; Fastlane <https://docs.fastlane.tools>; TestFlight's tester limits and 90-day build expiry <https://developer.apple.com/testflight/>; the Play Store internal track <https://support.google.com/googleplay/android-developer>.
- **[P2]** Tests and telemetry — Patrol <https://github.com/leancodepl/patrol>, Maestro <https://maestro.dev>,

Playwright <https://playwright.dev>, Vitest <https://vitest.dev>, pytest <https://docs.pytest.org>; Sentry <https://docs.sentry.io/platforms/flutter/>, PostHog <https://posthog.com/docs>, OpenTelemetry <https://opentelemetry.io/docs/>, and Lighthouse's Core Web Vitals definitions <https://developer.chrome.com/docs/lighthouse/performance/>.

15.6 Market and corpus evidence (§5, §11)

- **[E1]** State of JavaScript 2025 <https://2025.stateofjs.com> — build-tool and framework adoption shares, quoted in §8.1 and §10.8.
- **[E2]** Stack Overflow Developer Survey 2025 <https://survey.stackoverflow.co/2025/technology> — language and framework usage ranks, behind the C8 corpus argument in §7.1 and the Angular/Vue comparison in §8.3.
- **[E3]** GitHub Octoverse 2025 <https://octoverse.github.com> — TypeScript's position by repository activity, the other half of C8.
- **[E4]** W3Techs usage surveys <https://w3techs.com/technologies> — the installed-base counterweight to survey self-selection, cited in §11 because it limits how much a “largest corpus” claim can carry.
- **[E5]** Web Almanac 2025, Core Web Vitals chapter <https://almanac.httparchive.org> — the pass-rate figures behind the T4 performance budget in §13.
- **[E6]** Package registries, read on 7 October 2026 and the authority for every pin in Table 10: <https://pub.dev>, <https://www.npmjs.com>, <https://pypi.org>.

15.7 Licences and mark attribution

- **[L1]** Licence texts relied on in Appendix C — MIT <https://opensource.org/license/mit>, BSD-3-Clause <https://opensource.org/license/bsd-3-clause>, Apache-2.0 <https://www.apache.org/licenses/LICENSE-2.0>, PostgreSQL <https://www.postgresql.org/about/licence/>, SQLite's public-domain dedication <https://sqlite.org/copyright.html>, MPL-2.0 <https://www.mozilla.org/en-US/MPL/2.0/>, EPL-2.0 <https://www.eclipse.org/legal/epl-2.0/>, AGPL-3.0 <https://www.gnu.org/licenses/agpl-3.0.html> (cited because §10.8 rejects a detector family on those terms) and the Business Source Licence <https://mariadb.com/bsl11/> (cited because a broker's open-source line moved under it).
- **[L2]** Brand marks — every logo here is the project's own official mark, taken from the publicly maintained icon set <https://simpleicons.org> or, where one exists, the project's press assets, used for identification and commentary only under nominative fair use; they are not endorsements and the trade marks belong to their owners. Appendix C records the per-mark source, licence and rasterisation pipeline.

[HONESTY NOTE] Two limits deserve stating rather than hiding. The version and price figures in §10.8 and §10.5 were read from primary pages on a single day; for a document with a twelve-week life they are a baseline, not a forecast, and Table 10 is where they get re-checked. Several “origin” paragraphs in §7 and §8 recount events — a 2011 Dart announcement, a 2015 acquisition, a 2019 endorsement — attested by the projects' own archives rather than by one canonical reference; where accounts differ, this document follows the project's own publication record.